

# Communication Diagram 메시지 호출 명세

## 1. 문서 목적

이 문서는 다음 4개의 Communication Diagram의 메시지 호출 순서, 호출자와 수신자, 매개변수, 반환형, 메시지의 의미 및 실제 동작 과정을 구현 가능한 수준으로 설명한다.

- [Start Trading and Stop Trading.png](#)
- [Buy and Sell.png](#)
- [Show Trade History Details.png](#)
- [CSV Export.png](#)
- 위 네 이미지를 묶은 [Communication\\_Diagram.pdf](#)

추론한 타입은 새로운 기능이나 새로운 collaboration을 제안하는 것이 아니라, 다이어그램의 기존 메시지를 코드 operation으로 표현하기 위한 계약이다.

**Phase 0 정책 잠금:** 2026-08-20에 [Design/Architecture/Decisions/ADR-001~ADR-005](#)를 Accepted로 확정했다. 본 문서와 ADR이 함께 Communication 구현 계약을 이룬다. Phase 0은 production 동작 코드를 변경하지 않는다.

## 2. UML 표기법

### 2.1 Attribute

Attribute는 다음 형식으로 표기한다.

이름 : 타입 = 초기값 {속성}

- 이름은 attribute 이름이다.
- 타입은 attribute가 보관하는 값의 타입이다.
- = 초기값은 초기값이 있을 때만 적고, 초기값이 없으면 생략할 수 있다.
- {속성}은 {not null}, {unique}, {readOnly}처럼 값의 제약이나 특성을 적는다.

예:

```
context : TradingContext {not null}
filledQuantity : Decimal = 0 {not null}
exchangeOrderId : Long? = null
```

### 2.2 Operation

Operation은 다음 형식으로 표기한다.

이름(parameter list) : return type

- parameter는 `parameterName : Type` 형식으로 적는다.
- parameter가 여러 개면 쉼표로 구분한다.
- 기본값이 있으면 `parameterName : Type = defaultValue`로 적는다.
- nullable 값은 `Type?`으로 적는다.
- 반환값이 없으면 `void`를 사용한다.
- 생성 메시지도 이 문서에서는 반환형을 생략하지 않고 `Order(...) : Order`처럼 적는다.

예:

```
executeTrade(order : Order) : bool
find(query : TradeHistoryQuery) : List<Trade>
stopTrading() : void
```

### 2.3 UI Boundary와 HTML/CSS/JavaScript 구현의 관계

이 문서의 [AppShellUI](#), [RecentOrderUI](#), [TradeHistoryUI](#), [PopupUI](#)는 UML의 논리적인 `<<boundary>>` classifier이다. 사용자와 시스템 사이에서 입력을 받고 결과를 표시하는 역할을 표현하며, JavaScript의 `class` 문법이나 CSS의 class selector를 뜻하지 않는다. 이름에 `UI`가 포함되어더라도 `UIStateController`와 `UISTM`은 화면 Boundary가 아니라 UI 흐름과 상태를 제어하는 요소이다.

웹 구현에서 각 UML 요소는 다음과 같이 해석한다.

| UML 표현                                                               | HTML/CSS/JavaScript에서의 대응                                    | 의미                         |
|----------------------------------------------------------------------|--------------------------------------------------------------|----------------------------|
| <code>&lt;&lt;boundary&gt;&gt;</code><br><code>TradeHistoryUI</code> | HTML 구조 + CSS 스타일 + JavaScript/TypeScript UI 모듈 또는 component | 거래 내역 화면이라는 하나의 논리적인 UI 역할 |

| UML 표현                     | HTML/CSS/JavaScript에서의 대응                   | 의미                                                                                 |
|----------------------------|---------------------------------------------|------------------------------------------------------------------------------------|
| Boundary의 Attribute        | JavaScript가 실제로 보관하는 화면 상태, 선택값 또는 표시 데이터   | 예: 선택 기간, 선택 side, 현재 표시 중인 거래 목록                                                  |
| User -> UI Operation       | click, change, submit 등의 DOM event listener | 예: <code>selectFilter(...)</code> , <code>clickCSVExport()</code>                  |
| Controller -> UI Operation | DOM 갱신 또는 component render 함수               | 예: <code>displayTradeDetails(...)</code> , <code>displayFilteredTrades(...)</code> |
| CSS class                  | UML Class와 직접 대응하지 않음                       | HTML 요소를 선택하고 스타일을 적용하기 위한 selector                                                |

HTML과 CSS만으로는 이 문서의 Operation을 실행할 수 없다. 사용자 event 처리, Controller 호출 및 화면 갱신은 JavaScript/TypeScript의 event handler나 render 함수가 담당한다. 이때 반드시 `class TradeHistoryUI { ... }`와 같은 ES class로 구현할 필요는 없으며, 일반 함수, module 또는 framework component가 같은 Boundary 계약을 구현해도 된다.

이 문서의 클래스 목록은 분석·설계 수준에서 UI의 책임과 호출 계약을 보여준다. 따라서 클래스 다이어그램에는 위 UI 요소를 `<<boundary>>`로 표시한다. 구현 수준 클래스 다이어그램을 별도로 작성할 때는 JavaScript UI 모듈이나 component가 실제로 소유하는 상태만 Attribute로 남기고, Controller가 소유하는 상태는 해당 Controller에 배치해야 한다. `TradeHistoryUI -> trade-history.html + trade-history.css + trade-history.js`와 같은 실제 파일 구성은 필요하면 별도의 Component Diagram으로 표현한다.

### 3. 공통 타입과 원본 표기 정합성

#### 3.1 공통 타입

아래 이름은 매개변수와 반환형을 명확하게 하기 위한 값 타입 또는 결과 타입이다. 별도의 Communication Diagram 참여 클래스를 새로 제안하는 것이 아니다.

| 타입                                                 | 의미                                                                                          |
|----------------------------------------------------|---------------------------------------------------------------------------------------------|
| Decimal                                            | 가격, 수량, 금액, 비율처럼 부동소수 오차를 피해야 하는 수치                                                         |
| Instant / LocalDate                                | 거래 시각 / UI 날짜                                                                               |
| Path                                               | 로컬 파일 또는 디렉터리 경로                                                                            |
| Interval                                           | 1m, 30m, 4h, 1d Kline 주기                                                                    |
| Kline                                              | symbol, interval, openTime, OHLCV를 가진 봉 데이터                                                 |
| RegimeType                                         | 사용자가 선택하거나 RegimeSTM이 추천하는 REGIME 타입                                                        |
| RegimeEvent                                        | 최초 평가, 확정 4H 봉 마감 또는 EVALUATION_READY를 나타내는 불변 event                                        |
| RegimeEvaluationContext                            | 같은 MarketSnapshot version에서 파생한 slope, swing, current price와 live EMA9의 불변 묶음               |
| RegimeActionRequest                                | RegimeSTM이 결정하고 RegimeController가 수행하는 StartRegimeEvaluation 또는 ApplyRecommendedRegime 값    |
| RegimeSTMResult                                    | Regime 전이 ID, 전후 상태와 ordered Action 요청을 담은 결정 결과                                            |
| HistoryPeriod                                      | 거래 상세 조회 기간: TODAY, LAST_7_DAYS, LAST_30_DAYS, ALL                                          |
| TradeSide                                          | 거래 방향 필드: ALL, BUY, SELL                                                                    |
| CSVPeriod                                          | CSV 범위 preset: TODAY, WEEKLY, MONTHLY, CUSTOM                                               |
| CSVExportStatus                                    | CSV 내보내기 상태: IDLE, VALIDATING, EXPORTING, SUCCEEDED, FAILED                                 |
| UIEvent, UITransitionResult                        | UI STM에 전달되는 이벤트와 전이 결과                                                                     |
| TradingEvent, TradingContextView, TradingSTMResult | Trading STM에 전달되는 구체 event, 불변 Context snapshot과 전이/action 결과                               |
| PositionSnapshot                                   | Phase 7 start/stop Guard가 읽는 authoritative 수량과 평균 진입가의 불변 snapshot                          |
| TradingLogicSelectionResult                        | 사용자 선택 REGIME, 자원 상태와 commit된 Context version을 담은 application 결과                            |
| SplitRatioResult                                   | 적용된 scale-in/out Decimal과 commit된 Context version을 담은 application 결과                        |
| TradingSessionResult                               | session status, session ID, Context version과 STM transition/Action trace를 담은 application 결과 |
| AccountSnapshot                                    | REST 또는 user-data stream에서 정규화한 계좌 잔액 snapshot                                              |
| OrderResult                                        | 주문 상태와 fill을 정규화한 Gateway 결과                                                                |
| ExecutionSummary                                   | 한 주문의 여러 fill을 합친 체결 수량·금액·평균가·수수료 요약                                                       |
| RealizedResult                                     | 매도 체결의 실현손익 및 수익률 결과                                                                        |
| TradeDetailsResult                                 | 거래 목록, 보유량, 성과를 묶은 상세 화면 표시 결과                                                              |
| ValidationResult                                   | CSV option 검증 성공 여부와 오류 항목/사유                                                               |
| CSVExportResult                                    | CSV 생성 성공 여부, 생성 경로 또는 실패 사유                                                                |
| Subscription                                       | WebSocket 구독 handle                                                                         |

#### 3.2 원본 표기 정합성

아래 항목은 기능 차이가 아니라 다이어그램의 철자 또는 이름 정합성 문제다. 메시지 설명과 클래스 목록에서는 구현 가능한 정상 철자를 사용하되, 각 표의 설명에 원본 철자를 남긴다.

| 원본 표기                                     | 문서의 정규 표기                                                   | 비고                                                                       |
|-------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------|
| hadle(event)                              | handle(event)                                               | Buy and Sell 메시지 1                                                       |
| fetchSelectedTardingLogic()               | fetchSelectedTradingLogic(...)                              | Start/Stop 메시지 6.1.1.1                                                   |
| get 30mKlines()                           | get30mKlines(...)                                           | Start/Stop 메시지 1.2.2                                                     |
| VPP 내부 모델명<br>recordOrderExcution()       | recordOrderExecution(...)                                   | Buy and Sell 메시지 13의 PNG/PDF 표기를 우선함                                     |
| tradeHisoryFilterChanged()                | tradeHistoryFilterChanged(...)                              | History 메시지 2.1                                                          |
| TradeHostoryUI                            | TradeHistoryUI                                              | History PNG의 lifeline 이름                                                 |
| TradingHistoryUI                          | TradeHistoryUI                                              | CSV PNG의 lifeline 이름                                                     |
| WebsocketGateway                          | WebSocketGateway                                            | Buy and Sell PNG의 표기이며, Start/Stop PNG의 <b>WebSocketGateway</b> 와 같은 클래스 |
| setRegimeType, update,<br>setSaveLocation | 각각 setRegimeType(...), update(...),<br>setSaveLocation(...) | 원본에는 괄호가 없음                                                              |

TradingController와 초안의 TradingCoordinator, UISTM과 초안의 UIStateMachine, APiGateway와 초안의 BinanceAPiGateway처럼 이름이 충돌하는 경우에는 다이어그램의 TradingController, UISTM, APiGateway를 사용한다. WebSocket Gateway는 Start/Stop의 WebSocketGateway를 통합 클래스명으로 사용하되, Buy and Sell 메시지 설명에서는 그 그림의 WebsocketGateway 표기를 그대로 남긴다.

Operation 표는 UML 표기이므로 기존 다이어그램의 camelCase를 보존한다. Python 구현은 CODING\_CONVENTIONS.md에 따라 calculate\_4h\_indicators, order\_finished처럼 snake\_case를 사용하며, 같은 동작을 camelCase alias로 중복 구현하지 않는다.

4. Case 1 - Start Trading and Stop Trading

이 다이어그램의 최상위 순서는 다음과 같다.

1. 1~5: 앱 구동에 필요한 시장 데이터, 계좌, 거래 이력, UI 상태 및 interface 초기화
2. 6: 사용자의 REGIME 선택
3. 7: 사용자의 자동매매 시작 확인
4. 8: 사용자의 자동매매 중지 확인

4.1 시장 데이터 초기화 및 REGIME 추천

| 번호    | 호출자 -> 수신자                                  | Operation                                                                           | Parameter                           | Return         | 설명                                                                | 동작 과정                                                                |
|-------|---------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------|----------------|-------------------------------------------------------------------|----------------------------------------------------------------------|
| 1     | UIStateController -> MarketDataController   | InitializeMarketData(symbol : String = "ETHUSDT") : MarketSnapshot                  | symbol                              | MarketSnapshot | 시장 데이터 초기화<br>전체 흐름을 시작한<br>다.                                    | 1.1에서 실시간<br>시작한 뒤 1.20<br>하고, 1.3에서<br>함한다. 이어서<br>지표와 추천 RE        |
| 1.1   | MarketDataController<br>-> WebSocketGateway | startAllKlineBuffering(symbol : String, intervals : Set<Interval>) : Subscription   | 거래 symbol,<br>1m/30m/4h/1d 주기<br>집합 | Subscription   | REST 조회 중 발생<br>하는 최신 붐을 잃지<br>않도록 WebSocket<br>버퍼링을 먼저 시작한<br>다. | Gateway는 1.<br>든 뒤 수신 Kline<br>확용 buffer에<br>독 handle은 별<br>그리지 않는다. |
| 1.1.1 | WebSocketGateway -> Binance WebSocket       | subscribeAllKlineStreams(symbol : String, intervals : Set<Interval>) : Subscription | 거래 symbol, 주기 집합                    | Subscription   | Binance의 모든 대<br>상 Kline stream을<br>구독한다.                         | 구독 뒤 Binanc<br>는 붐을 Gatewa<br>interval, or<br>내부 Kline으로             |
| 1.2   | MarketDataController<br>-> APiGateway       | loadAllKlines(symbol : String, limit : int) : Map<Interval, List<Kline>>            | 거래 symbol, 주기별 조<br>회 개수            | 주기별 과거 붐       | 모든 대상 주기의 과<br>거 붐을 한 번에 준비<br>한다.                                | 1.2.1~1.2.4<br>환값을 주기별 N                                             |
| 1.2.1 | APiGateway -> Binance REST API              | get1mKlines(symbol : String, limit : int) : List<Kline>                             | 거래 symbol, 조회 개수                    | 1분봉 목록         | 1분봉 과거 데이터를<br>조회한다.                                              | Binance 응답을<br>정규화해 1.2의<br>한다.                                      |
| 1.2.2 | APiGateway -> Binance REST API              | get30mKlines(symbol : String, limit : int) : List<Kline>                            | 거래 symbol, 조회 개수                    | 30분봉 목록        | 30분봉 과거 데이터<br>를 조회한다.                                            | 원본 라벨은 공백<br>30mKlines()<br>할 수 있도록<br>get30mKline<br>화했다.           |
| 1.2.3 | APiGateway -> Binance REST API              | get4hKlines(symbol : String, limit : int) : List<Kline>                             | 거래 symbol, 조회 개수                    | 4시간봉 목록        | 4H REGIME 계산에<br>필요한 4시간봉을 조<br>회한다.                              | 확정봉과 현재 전<br>받아 이후 Regi<br>구분한다.                                     |

| 번호    | 호출자 -> 수신자                               | Operation                                                                                                           | Parameter                                                            | Return                | 설명                                         | 동작 과정                                                                                                                                          |
|-------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-----------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.2.4 | APIGateway -> Binance REST API           | get1dKlines(symbol : String, limit : int) : List<Kline>                                                             | 거래 symbol, 조회 개수                                                     | 일봉 목록                 | 일봉 과거 데이터를 조회한다.                           | 반환값을 다른 각 1.2의 결과에 3                                                                                                                           |
| 1.3   | MarketDataController -> MarketSnapshot   | update(klines : Map<Interval, List<Kline>>) : void                                                                  | REST 봉과 초기 WebSocket buffer를 병합한 결과                                  | void                  | 최신 시장 snapshot 을 갱신한다.                     | 같은 symbol, openTime 봉을 WebSocket 값 시간순 정렬과 붙인다.                                                                                                |
| 1.4   | MarketDataController -> RegimeController | calculate4HIndicators(snapshot : MarketSnapshot) : IndicatorSnapshot                                                | 최신 시장 snapshot                                                       | 계산된 지표 snapshot       | REGIME 판정용 4H 지표를 계산한다.                    | 확정 4H 봉과 존 분리한다. 확정봉 열, 최근 6개 EA와 HH/HL/LH/L 진행 중인 봉으로 계산한다.                                                                                   |
| 1.4.1 | RegimeController -> IndicatorSnapshot    | update(ema9Series : List<Decimal>, ema9Slope : Decimal, swingStructure : SwingStructure, liveEma9 : Decimal) : void | 계산한 4H 지표                                                            | void                  | 계산 결과를 동일 평가 시점의 IndicatorSnapshot 에 반영한다. | 원본에는 update 없다. 이 snapshot STM 입력이 된다                                                                                                          |
| 1.5   | MarketDataController -> RegimeController | recommendRegime(indicators : IndicatorSnapshot) : RegimeType                                                        | 4H IndicatorSnapshot                                                 | 추천 REGIME             | 현재 4H 데이터에 대한 추천 REGIME을 요청한다.             | RegimeController MarketSnapshot RegimeEvaluation 을 준비하고 1.1 microstep 결과다. ApplyRecommend 이 성공한 뒤 fact 환하며 추천값은 덮어쓰지 않는다                       |
| 1.5.1 | RegimeController -> RegimeSTM            | handle(event : RegimeEvent, context : RegimeEvaluationContext?) : RegimeSTMResult                                   | 최초/4H 마감 event와 optional Context, 이어지는 EVALUATION_READY 와 필수 Context | 전이와 ordered Action 요청 | Regime STM의 한 판정 microstep을 실행한다.          | 첫 호출은 EA-0101~EA-1055 StartRegime 반환한다. Context 을 수행한 뒤 같은 evaluation EVALUATION_ Context로 동일 시 호출하고, EA 과 ApplyRecommend 을 수행한다. ST 직접 변경하지 않 |

초기 snapshot을 게시한 뒤에는 같은 Kline 구독을 끊었다가 다시 만드는 대신 buffer 소유권을 live observer로 원자 전환한다. 이 Phase 13 연속 경로는 다음 메시지를 사용한다.

| 번호 | 호출자 -> 수신자                               | Operation                                                                                                | Parameter                              | Return                 | 설명                                            | 동작 과정                                                                                                                                                                          |
|----|------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------|------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1L | MarketDataController -> WebSocketGateway | promoteKlineBufferToLive(observer : Callable<Kline>, subscription : Subscription? = null) : Subscription | 정규화 Kline observer와 optional 현재 handle | 같은 세대 의 live 구독 handle | 초기화 buffer와 지속 stream 사이의 손실 없는 소유권 전환을 요청한다. | Gateway lock 아래 남은 buffer를 순서대로 전달한 뒤 동일 generation의 후속 Kline을 observer로 보낸다. 새 구독을 열어 시간 공백을 만들지 않는다. stale generation과 중복 (symbol, interval, openTime, eventTime)은 전달하지 않는다. |

| 번호   | 호출자 -> 수신자                                | Operation                                                                                                                   | Parameter                                   | Return                                      | 설명                                                   | 동작 과정                                                                                                                                                                                                                                                                                                                                              |
|------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|---------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1L.1 | WebSocketGateway -> MarketDataController  | observeKline(kline : Kline) : void                                                                                          | 정규화된 현재 generation Kline                    | void                                        | 지속 시장 event를 application owner에 전달한다.                | Controller는 interval별 단조 event time과 open time을 검사한다. UTC 4H 경계에서는 (1m close, 30m close, 4H close, next 4H open) 네 source를, UTC 자정에는 여기에 (1D close, next 1D open)을 더한 여섯 source를 bounded slot에 모아 유효 arrival order와 무관하게 한 번만 게시한다.                                                                                                                |
| 1L.2 | MarketDataController -> MarketSnapshot    | mergeKline(kline : Kline) : MarketSnapshot                                                                                  | 중복 제거와 연속성 검사를 통과한 Kline                    | 새 version의 시장 snapshot                      | 현재 봉 교체 또는 새 봉 추가를 원자 반영한다.                          | REST/WS 초기화와 같은 symbol-interval-openTime 규칙을 사용한다. 경계 tuple 전체가 모이기 전에는 이전 snapshot을 유지하며 overflow, boundary mismatch, upper open-before-close와 조기 event time은 publication 없이 fail closed하고 full resync를 요청한다.                                                                                                                                     |
| 1L.3 | MarketDataController -> TradingController | observeMarketEvaluation(evaluation : MarketEvaluationSnapshot, marketVersion : int, sourceEventId : String) : TradingEvent? | 같은 snapshot version에서 계산한 전략 지표와 provenance | queue identity가 부여된 event 또는 비활성. 중복이면 null | private Action 호출 없이 시장 평가를 Trading event path에 넣는다. | TradingController는 immutable evaluation/version을 event에 결합해 직렬 queue에 넣고 bounded worker를 깨운다. Worker가 해당 event를 claim한 뒤 queue preparer가 그 evaluation으로 Context market를 갱신하고 lower/new-30m/upper/market-updated 중 정확한 event로 재분류한다. 뒤 Kline이 먼저 관찰돼도 앞 event가 뒤 Context로 처리되지 않는다. 4H 확정봉이면 RegimeController 재평가도 수행하지만 사용자가 선택한 REGIME은 변경하지 않는다. |

현재 Kline 세대의 disconnect, payload gap 또는 observer 실패는 Gateway가 한 번만 `markMarketStreamReconciliationRequired`로 알린다. Controller는 이전 snapshot provenance와 이미 관찰한 주문 결과를 보존하고 전략 scheduler를 비운 뒤, 새 stream 세대와 전체 REST Kline을 다시 결합해 같은 version의 MarketSnapshot과 REGIME 평가를 만든다. 복구 순서는 새 generation buffering → 전체 REST 조회 → 동일 handle의 live promotion과 buffer drain → REST/WS를 한 MarketSnapshot version으로 병합 → `reconcileRegime(snapshot)`이다. 새 4H source candle이면 정상 재평가하고, 동일 candle이면 지표와 기존 추천-STM state를 검증한 뒤 STM transition을 반복하지 않고 새 `sourceMarketVersion`에 결과만 재결합한다. `RegimeResult.sourceMarketVersion`, authoritative `MarketSnapshot.version`, completion version과 Gateway live readiness가 모두 일치한 뒤에만 `completeMarketStreamReconciliation`이 시장 source blocker를 해제한다. `None`, version mismatch 또는 평가 실패면 새 구독을 달고 시장 gate를 계속 차단한다. 장애 당시 `RUNNING`이던 세션의 중단 provenance는 이 성공으로 지우지 않는다. 같은 주문의 cancel/query와 terminal history 복구는 허용하지만, terminal patch가 Context를 `IDLE`로 바꾸더라도 공개 status와 command gate는 operator 세션 재조정 전까지 `RECONCILIATION_REQUIRED`로 남는다.

#### 4.2 계좌 초기 snapshot 및 account stream

| 번호    | 호출자 -> 수신자                             | Operation                                                      | Parameter | Return           | 설명                                        | 동작 과정                                                                        |
|-------|----------------------------------------|----------------------------------------------------------------|-----------|------------------|-------------------------------------------|------------------------------------------------------------------------------|
| 2     | UIStateController -> TradingController | loadAccount(asset : String = "ETH") : Account                  | 조회할 기준 자산 | 초기화된 Account     | 계좌 초기 snapshot을 준비하고 실시간 계좌 stream을 시작한다. | 2.1의 REST snapshot을 내부 Account에 반영한 후 2.2로 변경 stream을 구독한다.                  |
| 2.1   | TradingController -> APIGateway        | fetchAccountSnapshot(asset : String = "ETH") : AccountSnapshot | 기준 자산     | 정규화된 계좌 snapshot | 최초 화면과 거래 context에 사용할 계좌 정보를 조회한다.       | 2.1.1의 Binance 응답을 잔액, ETH 보유량, 평가 정보로 정규화한다.                                |
| 2.1.1 | APIGateway -> Binance REST API         | getAccount() : BinanceAccountResponse                          | 없음        | Binance 계좌 응답    | Binance 계좌 정보 API를 호출한다.                  | 반환 응답은 <code>fetchAccountSnapshot(...)</code> 내부에서 소비하며 별도 메시지 답변을 추가하지 않는다. |
| 2.2   | TradingController ->                   | startAccountInfoStream() : Subscription                        | 없음        | 계좌 stream 구      | 최초 snapshot 이후의 잔액 변경                     | Gateway가 2.2.1을 통해 구독을 만들고 이후 account event를 정규화한다.                          |

| 번호    | 호출자 -> 수신자                            | Operation                             | Parameter | Return    | 설명                                      | 동작 과정                                                                                                                                                                                                                                                |
|-------|---------------------------------------|---------------------------------------|-----------|-----------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       | WebSocketGateway                      |                                       |           | 독 handle  | 을 받을 user-data stream을 시작한다.            |                                                                                                                                                                                                                                                      |
| 2.2.1 | WebSocketGateway -> Binance WebSocket | subscribeAccountInfo() : Subscription | 없음        | 구독 handle | Binance account/user-data stream을 구독한다. | 구독 자체의 응답은 함수 반환값이다. 이후 비 동기 event는 bounded 단일 FIFO dispatcher가 수신 loop와 분리해 처리한다. enqueue부터 callback 완료까지 <b>accountReady = false</b> 이고 overflow-consumer failure-disconnect 는 구독을 닫고 full reconciliation을 요구한다. 이 초기 구독 메시지는 reply 화살표를 추가하지 않는다. |

#### 4.3 거래 이력과 성과 로드

| 번호    | 호출자 -> 수신자                                       | Operation                                         | Parameter   | Return            | 설명                              | 동작 과정                                                                |
|-------|--------------------------------------------------|---------------------------------------------------|-------------|-------------------|---------------------------------|----------------------------------------------------------------------|
| 3     | UIStateController -> TradeHistoryController      | loadTradeHistory() : TradeHistory                 | 없음          | 초기화된 거래 이력        | 저장된 거래 이력과 누적 성과를 메모리에 준비한다.    | 3.1에서 저장 데이터를 가져오고 3.2의 TradeHistory와 3.3의 Performance를 초기화한다.       |
| 3.1   | TradeHistoryController -> TradeHistoryRepository | getTradeHistory() : List<Trade>                   | 없음          | 저장된 거래 목록         | repository에 저장된 모든 Trade를 요청한다. | 3.1.1에서 파일을 읽고 역직렬화한 목록을 반환한다.                                       |
| 3.1.1 | TradeHistoryRepository -> Local File System      | read(path : Path) : String                        | 거래 이력 저장 경로 | 직렬화된 파일 내용        | 로컬 거래 이력 파일을 읽는다.               | 파일 내용의 parsing과 List<Trade> 변환은 repository 내부 처리이므로 별도 메시지를 만들지 않는다. |
| 3.2   | TradeHistoryController -> TradeHistory           | TradeHistory(trades : List<Trade>) : TradeHistory | 읽은 거래 목록    | TradeHistory 인스턴스 | 인메모리 거래 이력 entity를 생성 또는 초기화한다. | 이후 주문 기록, 상세 조회 및 필터의 공통 원천으로 사용한다.                                  |
| 3.3   | TradeHistoryController -> Performance            | Performance(trades : List<Trade>) : Performance   | 읽은 거래 목록    | Performance 인스턴스  | 기존 거래를 기준으로 누적 성과를 복원한다.        | 실현손익, 수수료, 누적 수익률과 매도 성과를 계산해 이후 조회와 신규 Trade 반영에 사용한다.              |

#### 4.4 UI STM 및 interface 시작

| 번호 | 호출자 -> 수신자                      | Operation                                                      | Parameter     | Return      | 설명                              | 동작 과정                                                                           |
|----|---------------------------------|----------------------------------------------------------------|---------------|-------------|---------------------------------|---------------------------------------------------------------------------------|
| 4  | UIStateController -> UISTM      | run(initialEvent : UIEvent = APP_STARTED) : UITransitionResult | 앱 시작 UI event | 초기 UI 전이 결과 | UI STM을 초기 상태에서 실행한다.           | 화면, 팝업, 필터 등 UI 상태의 초기 전이를 결정한다. 외부 I/O는 UISTM이 직접 수행하지 않는다.                    |
| 5  | UIStateController -> AppShellUI | startInterface() : void                                        | 없음            | void        | AppShell을 표시하고 사용자 입력 수신을 시작한다. | 앞 단계에서 준비된 추천 REGIME, 계좌, 이력, 성과 및 trading 상태를 controller의 UI binding을 통해 표시한다. |

#### 4.5 사용자 REGIME 선택

| 번호  | 호출자 -> 수신자                      | Operation                                    | Parameter       | Return | 설명                            | 동작 과정                                                    |
|-----|---------------------------------|----------------------------------------------|-----------------|--------|-------------------------------|----------------------------------------------------------|
| 6   | User -> AppShellUI              | selectRegime(regimeType : RegimeType) : void | 사용자가 선택한 REGIME | void   | 사용자가 실제 거래에 적용할 REGIME을 선택한다. | 추천 REGIME과 별개의 사용자 선택값이며 Boundary가 6.1로 전달한다.            |
| 6.1 | AppShellUI -> UIStateController | selectRegime(regimeType : RegimeType) : void | 선택 REGIME       | void   | UI 입력을 제어 계층에 전달한다.           | UIStateController는 6.1.1을 통해 RegimeController에 선택을 적용한다. |

| 번호        | 호출자 -> 수신자                                  | Operation                                                                                                       | Parameter                                           | Return                              | 설명                                        | 동작 과정                                                                                                                                                                                                                                                                    |
|-----------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|-------------------------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6.1.1     | UIStateController<br>-><br>RegimeController | setRegimeType(regimeType : RegimeType, commandId : String, expectedVersion : int) : TradingLogicSelectionResult | 선택 REGIME, 먹 등 command ID, 호출자가 관측한 Context version | 선택값, 지원 상태와 commit된 Context version | 추천값이 아닌 사용자의 적용 REGIME을 설정한다.             | 원본 라벨에는 괄호가 없다. Phase 7 concrete application Operation은 transport의 <b>Idempotency-Key</b> 와 <b>commandId</b> 로 전달하고 optimistic version을 검증한다. active trading session에서는 <b>TRADING_ACTIVE</b> 거부하고 기존 선택을 유지한다. 정지 태에서는 선택값과 지원 상태를 보존한 <b>6.1.1.1</b> 로 거래 logic을 조회한다. |
| 6.1.1.1   | RegimeController<br>-><br>TradingController | fetchSelectedTradingLogic(regimeType : RegimeType) : TradingSTM                                                 | 선택 REGIME                                           | 선택된 TradingSTM 또는 typed failure     | 선택 REGIME에 맞는 trading logic을 요청한다.        | 원본 철자는 <b>fetchSelectedTradingLogic</b> 이다. <b>TYPE_0</b> 은 <b>SUPPORTED/LOWER_BB/READY</b> 상태 BB에서 <b>SAFE_TERMINATION</b> 사용한다. <b>TYPE_1~TYPE_4</b> 는 <b>UNSUPPORTED_TRADING_LOGIC</b> 이며 fallback은 없다.                                                               |
| 6.1.1.1.1 | TradingController<br>-> TradingSTM          | getSTMInstance(regimeType : RegimeType) : TradingSTM                                                            | 선택 REGIME                                           | 해당 TradingSTM 인스턴스 또는 typed failure | 선택된 logic을 실행할 session 전용 STM 인스턴스를 가져온다. | ADR-001의 불변 mapping을 엄격히 적용한다. <b>TYPE_0</b> 만 정확히 109개 lower-BB transition이 있는 새 session 인스턴스를 받고, 미지원 타입은 인스턴스를 만들지 않는다.                                                                                                                                               |

#### 4.6 자동매매 시작

아래 표는 필수 누락 메시지를 삽입한 최종 numbering이다. 기존 그림의 **7.1.1.1: run()**은 **7.1.1.2**로 이동해야 한다.

| 번호      | 호출자 -> 수신자                                   | Operation                                                                                                                                       | Parameter                                           | Return                                                 | 설명                                 | 동작 과정                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7       | User -> AppShellUI                           | startConfirmed() : void                                                                                                                         | 없음                                                  | void                                                   | 사용자가 시작 확인 팝업에서 거래 시작을 확정한다.       | Boundary가 <b>7.1</b> 로 확인 event를 전달한다.                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 7.1     | AppShellUI -> UIStateController              | startTrading() : void                                                                                                                           | 없음                                                  | void                                                   | 자동매매 시작 UI event를 전달한다.            | UI 상태 전이 후 실제 trading 시작을 <b>7.1.1</b> 에 위임한다.                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 7.1.1   | UIStateController<br>-><br>TradingController | startTrading(commandId : String, expectedVersion : int) : TradingSessionResult                                                                  | 먹등 command ID와 호출자가 관측한 Context version             | session status, session ID, commit된 version과 STM trace | TradingController에 자동매매 시작을 요청한다.  | Controller가 selected REGIME, 지원 mapping, connection, Account/Position과 실행 mode gate를 먼저 검증한다. 미지원이면 Context를 초기화하지 않고 <b>UNSUPPORTED_TRADING_LOGIC</b> 으로 거부한다. <b>fake</b> 는 명령을 허용하고, Phase 9 <b>testnet</b> 은 고정 Testnet endpoint-startup reconciliation·별도 주문 opt-in·양수 BUY entry max-notional을 모두 만족할 때만 허용한다. <b>disabled</b> , read-only <b>testnet</b> , <b>live</b> 는 fail closed다. 성공 시 <b>7.1.1.1</b> 과 <b>7.1.1.2</b> 를 순서대로 정확히 한 번 실행하며, 같은 <b>commandId</b> 와 payload는 최초 typed 결과를 재사용한다. |
| 7.1.1.1 | TradingController<br>-> TradingContext       | initialize(account : Account, selectedRegime : RegimeType, position : PositionSnapshot, scaleInRatio : Decimal, scaleOutRatio : Decimal) : void | 계좌, 선택 REGIME, 현재 authoritative 포지션 snapshot, 분할 비율 | void                                                   | TradingSTM이 사용할 시작 context를 초기화한다. | Phase 7 canonical Start/Stop 모델은 기존 클래스인 <b>TradingContext</b> lifeline을 이 위치에 포함한다. <b>positionOwner</b> , pending 주문 값, <b>tradingPhase</b> , <b>lowerEventId</b> 등 runtime 값을 일관된 시작값으로 만들고 계좌·포지션·설정 참조를 연결한다.                                                                                                                                                                                                                                                                                        |
| 7.1.1.2 | TradingController<br>-> TradingSTM           | run(context : TradingContextView) : TradingSTMResult                                                                                            | 초기화된 TradingContext                                 | 초기 trading action                                      | TradingSTM을 실행한다.                  | 원본 그림의 번호는 <b>7.1.1.1</b> 이다. Controller가 mutable Context를 snapshot으로 만든 뒤 넘긴다. STM은 시                                                                                                                                                                                                                                                                                                                                                                                                                    |

| 번호 | 호출자 -> 수신자 | Operation | Parameter     | Return | 설명 | 동작 과정                                             |
|----|------------|-----------|---------------|--------|----|---------------------------------------------------|
|    |            |           | 에서 만든 불변 view |        |    | 작 가능 조건과 초기 상태를 결정하고 반환 Action은 Controller가 소비한다. |

#### 4.7 자동매매 중지, 복구 포지션 인수와 전량 매도

아래 표는 필수 누락 메시지와 안전 branch를 삽입한 최종 numbering이다. 기존 그림의 무조건 전량 매도 흐름은 **Position.quantity**와 pending 주문을 확인하는 **alt** fragment로 교체해야 한다. **a**는 무포지션, **p**는 pending reconciliation, **b**는 보유 포지션 force-sell branch를 뜻한다. 재시작에서 복원된 포지션은 일반 stop의 의미를 넓히지 않고 별도 **8R** 명시 청산 Operation으로만 인수한다.

| 번호         | 호출자 -> 수신자                             | Operation                                                                                               | Parameter                                                 | Return                                            | 설명                                           |
|------------|----------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|---------------------------------------------------|----------------------------------------------|
| 8          | User -> AppShellUI                     | stopConfirmed() : void                                                                                  | 없음                                                        | void                                              | 사용자가 자동 매매 중지를 확정한다.                         |
| 8.1        | AppShellUI -> UIStateController        | stopTrading() : void                                                                                    | 없음                                                        | void                                              | 중지 확인 event를 제어 계층에 전달한다.                    |
| 8.1.1      | UIStateController -> TradingController | stopTrading(commandId : String, expectedVersion : int) : TradingSessionResult                           | 명등 command ID와 호출자가 관측한 Context version                   | terminated, stopping 또는 reconciliation session 결과 | 신규 진입 차단과 필요한 포지션 정리를 포함한 중지 절차를 시작한다.       |
| 8.1.1.1    | TradingController -> TradingSTM        | handle(event : TradingEvent, context : TradingContextView) : TradingSTMResult                           | event = STOP_CONFIRMED, 현재 불변 Context view                | 중지 action/전이 결과                                   | TradingSTM에 중지를 전달해 신규 진입을 차단하고 중지 절차를 시작한다. |
| 8.1.1.2a   | TradingController 내부 branch            | no sell operation                                                                                       | [Position.quantity == 0 && pendingOrder == null]          | LOGIC_TERMINATED                                  | 무포지션 중지를 완료한다.                               |
| 8.1.1.2p   | TradingController -> APIGateway        | queryOrderResult(symbol : String, orderId : Long? = null, clientOrderId : String? = null) : OrderResult | [pendingOrder != null], 기존 주문 식별자                         | 정규화 주문 결과                                         | pending 주문 사실을 먼저 확인한다.                      |
| 8.1.1.2p.1 | TradingController -> APIGateway        | cancelOrder(symbol : String, orderId : Long? = null, clientOrderId : String? = null) : OrderResult      | 취소 가능한 기존 주문 식별자                                          | 취소 요청 결과                                          | pending 주문의 잔여 체결을 중지한다.                     |
| 8.1.1.2b   | TradingController -> APIGateway        | sellAllPosition(symbol : String, quantity : Decimal) : OrderResult                                      | [Position.quantity > 0 && pendingOrder == null], 현재 잔여 수량 | 강제 매도 주문 결과                                       | 실제 잔여 포지션만 전량 매도한다.                          |
| 8.1.1.2b.1 | APIGateway -> Binance REST API         | sellAllPosition(symbol : String, quantity : Decimal) : BinanceOrderResponse                             | 보유 symbol과 0보다 큰 잔여 수량                                    | Binance 주문 응답                                     | Binance Spot에 실제 강제 매도 주문을 제출한다.             |

| 번호       | 호출자 -> 수신자                                   | Operation                                                                                                                                                                                                                              | Parameter                                                          | Return                                                  | 설명                                                                    |
|----------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------|
| 8.1.1.3  | TradingController<br>-> TradingSTM           | orderFinished(event : TradingEvent,<br>context : TradingContextView) :<br>TradingSTMResult                                                                                                                                             | FORCE_SELL_FINISHED<br>또는<br>FORCE_SELL_FAILED,<br>결과 반영 뒤 Context | 중지 완료/재조정 전이                                            | 보유 또는<br>pending<br>branch의 구<br>체 완료 결과<br>를 STM에 전<br>달한다.          |
| 8R       | User -><br>AppShellUI                        | recoveredPositionLiquidationConfirmed()<br>: void                                                                                                                                                                                      | 없음                                                                 | void                                                    | 사용자가<br>startup에서<br>복원된 포지션<br>의 명시 청산<br>을 확정한다.                    |
| 8R.1     | AppShellUI -><br>UIStateController           | liquidateRecoveredPosition() : void                                                                                                                                                                                                    | 없음                                                                 | void                                                    | 복구 포지션<br>청산 의도를<br>제어 계층에<br>전달한다.                                   |
| 8R.1.1   | UIStateController<br>-><br>TradingController | liquidateRecoveredPosition(commandId :<br>String, expectedVersion : int) :<br>TradingSessionResult                                                                                                                                     | 역등 command ID와 호출<br>자가 관측한 Context<br>version                     | stopping, terminated<br>또는 reconciliation<br>session 결과 | startup에서<br>설명 가능하게<br>복원된 포지션<br>을 청산 전용<br>세션으로 인수<br>한다.          |
| 8R.1.1.1 | TradingController<br>-> TradingContext       | initialize(account : Account,<br>recoveredRegime : RegimeType, position<br>: PositionSnapshot, scaleInRatio :<br>Decimal, scaleOutRatio : Decimal);<br>updatePosition(position :<br>PositionSnapshot, owner :<br>PositionOwner) : void | durable history에서 검증<br>한 REGIME-owner와<br>authoritative Position  | void                                                    | 복구<br>Position만<br>소유하는<br>liquidation-<br>only<br>Context를<br>초기화한다. |
| 8R.1.1.2 | TradingController<br>-> TradingSTM           | handle(event : TradingEvent, context :<br>TradingContextView) : TradingSTMResult                                                                                                                                                       | event =<br>STOP_CONFIRMED, 복구<br>owner가 설정된 불변<br>Context view     | G-06 force-sell 결과                                      | 자동 전략을<br>시작하지 않고<br>복구 포지션<br>청산만 시작하<br>다.                          |

Production bootstrap은 위 메시지를 새 업무 lifeline으로 확장하지 않고, process당 하나의 중단 가능한 event runtime worker가 `TradingController.runEventRuntimeCycle()`을 깨운다. 이 bounded Operation은 `RETRY_BACKOFF` due 작업을 한 번 release한 뒤 직렬 queue를 유한 microstep만큼 drain한다. 동기 REST 결과와 WebSocket 주문 결과는 worker wake만 요청하며, route별 drain, 주문별 timer/thread와 busy loop는 금지한다. worker는 cycle 뒤 authoritative trading snapshot을 transport observer에 게시하므로 동기 `FILLED`도 `8R.1.1`의 202 응답 뒤 수동 drain 없이 `8.1.1.3`과 `G-06F`를 완료할 수 있다.

8R.1.1.2가 기존 메시지 5와 6 사이의 제출 전 pending-order PREPARED journal을 저장할 때, 저장 Operation의 예외는 file-parent-directory fsync 이전 실패와 fsync 완료 뒤 실패를 구분하지 못하는 모호한 durability cut-point다. 이 경우 6/6.1의 REST POST는 금지하지만, 이미 durable할 수 있는 journal과 liquidation-only session을 NOT\_STARTED로 rollback하지 않는다. TradingController는 RECONCILIATION\_REQUIRED와 command/safe-shutdown gate를 유지하고 운영자가 sidecar와 같은 client ID의 거래소 사실을 조정하게 한다.

복구 청산에는 BUY entry cap을 적용하지 않으며 exchange 수량 step 때문에 여러 부분 주문으로 시작하지 않는다. 최초 prepare가 전량을 내림 조정하면 위 8R.1.1 명령 전 상태로 원자 복원하고 같은 command ID의 재시도를 허용한다. 실제 terminal partial fill 뒤 남은 수량의 prepare가 전량 조건을 만족하지 못하면 이미 발생한 거래소-history 사실을 되돌리지 않고 RECONCILIATION\_REQUIRED에서 운영자 개입을 요구한다.

## 5. Case 2 - Buy and Sell

### 5.1 흐름 분기

- 매수: 1~10 -> 12 -> 13~14; 매도 원가가 필요하지 않으므로 11은 생략한다.
- 매도: 1~10 -> 11 -> 12 -> 13~14.
- 최초 주문 결과가 FILLED이면 8, 8.1, 8.2, 9를 건너뛰고 10으로 진행한다.
- 최초 결과가 NEW, PARTIALLY\_FILLED, UNKNOWN이면 8 -> 8.1/8.2 -> 9 -> 10으로 진행한다.
- 주문 하나의 여러 fill은 10에서 하나의 ExecutionSummary로 집계한다.

### 5.2 메시지 호출

| 번호  | 호출자 -> 수신자                                            | Operation                                                                                                                               | Parameter                                                                                                | Return                                  | 설명                          |
|-----|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------|-----------------------------|
| 1   | TradingController -> TradingSTM                       | handle(event : TradingEvent, context : TradingContextView) : TradingSTMResult                                                           | 전략/시장 조건 event와 같은 평가 시점의 불변 Context view                                                                | STM action 결과                           | 전략                          |
| 2   | TradingController -> TradingContext                   | applyTradingSTMResult(result : TradingSTMResult) : void                                                                                 | 메시지 1의 결과                                                                                                | void                                    | STM 반영                      |
| 3   | TradingController -> TradingContext                   | getSplitRatio() : Decimal                                                                                                               | 없음                                                                                                       | 현재 주문 방향의 분할 비율                         | 이번 한도                       |
| 4   | TradingController -> claimed MarketEvaluationSnapshot | getRealtimePrice() : Decimal                                                                                                            | claim된 TradingEvent의 immutable evaluation/version                                                        | 주문 결정 시점의 30분 candidate 가격              | Put<br>환출<br>eval<br>을<br>되 |
| 5   | TradingController -> Order                            | Order(symbol : String, side : OrderSide, strategy : StrategyType, requestedQuantity : Decimal, marketPriceAtDecision : Decimal) : Order | 주문 의도와 결정 가격                                                                                             | 로컬 Order                                | 거래<br>려는                    |
| 5.1 | TradingController 내부 Operation                        | evaluateBuyRisk(order : Order) : RiskDecision                                                                                           | exchange filter로 실제 제출 수량이 확정된 BUY Order와 같은 lock에서 캡처한 Account/Position/Market/History/pending snapshot | 허용 또는 typed risk 차단과 RiskBudgetSnapshot | 모든<br>하는                    |
| 6   | TradingController -> APIGateway                       | submitOrder(order : Order) : OrderResult                                                                                                | 로컬 Order                                                                                                 | 정규화된 최초 주문 결과                           | 주문                          |
| 6.1 | APIGateway -> Binance REST API                        | placeOrder(symbol : String, side : OrderSide, quantity : Decimal) : BinanceOrderResponse                                                | symbol, side, 수량                                                                                         | Binance 최초 주문 응답                        | Bin                         |

| 번호     | 호출자 -> 수신자                                       | Operation                                                                                               | Parameter                                                       | Return       | 설명                      |
|--------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|--------------|-------------------------|
| 7      | TradingController -> Order                       | applyOrderResult(result : OrderResult) : void                                                           | 최초 주문 결과                                                        | void         | 최초 영한                   |
| 8      | TradingController -> APiGateway                  | queryOrderResult(symbol : String, orderId : Long? = null, clientOrderId : String? = null) : OrderResult | symbol과 거래소 또는 client 주문 ID                                     | 재조회한 주문 결과   | [s1<br>PAF<br>UNH<br>매도 |
| 8.1    | APiGateway -> Binance REST API                   | getOrderStatus(symbol : String, orderId : Long) : BinanceOrderStatusResponse                            | symbol, 주문 ID                                                   | 최신 주문 상태     | 최신 시                    |
| 8.2    | APiGateway -> Binance REST API                   | getAccountTrades(symbol : String, orderId : Long) : List<Fill>                                          | symbol, 주문 ID                                                   | 실제 fill 목록   | 해당 료를                   |
| 9      | TradingController -> Order                       | reapplyOrderResult(result : OrderResult) : void                                                         | 메시지 8의 재조회 결과                                                   | void         | 재조 적들                   |
| 10     | TradingController -> Order                       | buildExecutionSummary() : ExecutionSummary                                                              | 없음                                                              | 체결 요약        | Ord 결고                  |
| 11     | TradingController -> Position                    | getCostBasis(executedQuantity : Decimal) : Decimal                                                      | 실제 매도 체결 수량                                                     | 해당 수량의 취득원가  | [매 한                    |
| 12     | TradingController -> Position                    | applyExecution(summary : ExecutionSummary) : void                                                       | 체결 요약                                                           | void         | 실제                      |
| 13     | TradingController -> TradeHistoryController      | recordOrderExecution(order : Order, summary : ExecutionSummary, costBasis : Decimal?) : void            | Order, 체결 요약, 매도 시 원가                                           | void         | 완충 에                    |
| 13.1   | TradeHistoryController -> Performance            | calculateRealizedResult(summary : ExecutionSummary, costBasis : Decimal) : RealizedResult               | 매도 체결 요약, 취득원가                                                  | 이번 매도의 실현 결과 | 이번 한                    |
| 13.2   | TradeHistoryController -> Trade                  | Trade(order : Order, summary : ExecutionSummary, realizedResult : RealizedResult?) : Trade              | Order, 체결 요약, 선택적 실현 결과                                         | Trade        | 실제                      |
| 13.3   | TradeHistoryController -> TradeHistory           | addTrade(trade : Trade) : void                                                                          | 새 Trade                                                         | void         | 새 카 한                   |
| 13.4   | TradeHistoryController -> Performance            | applyNewTrade(trade : Trade) : void                                                                     | 새 Trade                                                         | void         | 새 가 에                   |
| 13.5   | TradeHistoryController -> TradeHistoryRepository | saveThisTradeByOrderID(orderId : Long, trade : Trade) : void                                            | 주문 ID, 새 Trade                                                  | void         | 주문 다.                   |
| 13.5.1 | TradeHistoryRepository -> Local File System      | write(path : Path, data : String) : void                                                                | 저장 경로, 직렬화된 Trade                                               | void         | 거리                      |
| 14     | TradingController -> TradingSTM                  | orderFinished(event : TradingEvent, context : TradingContextView) : TradingSTMResult                    | 전략·side·성공/실패가 명시된 concrete normalized outcome과 결과 반영 뒤 Context | 주문 결과 전이     | 주문 알                    |

TradingContext 생성/초기화는 주문 실행 이전인 Case 1의 자동매매 시작 흐름에 추가해야 하며, 이 case에 중복 추가하지 않는다.

6. Case 3 - Show Trade History Details

이 다이어그램은 두 흐름으로 나뉜다.

1. 1 계열: 최근 체결 영역에서 전체 보기를 선택해 상세 화면을 처음 표시
2. 2 계열: 상세 화면에서 기간/매수·매도 필터를 변경해 목록을 다시 표시

6.1 전체 거래 상세 화면 표시

| 번호      | 호출자 -> 수신자                                  | Operation                                                                                                 | Parameter                | Return                  | 설명                                       | 동작 과정                                                            |
|---------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------|-------------------------|------------------------------------------|------------------------------------------------------------------|
| 1       | User -> RecentOrderUI                       | showAllTradingDetails() : void                                                                            | 없음                       | void                    | 사용자가 최근 체결 패널의 전체 보기를 선택한다.              | RecentOrderUI가 상세 화면 진입 event를 1.1로 전달한다.                        |
| 1.1     | RecentOrderUI -> UIStateController          | showAllTradingDetails() : void                                                                            | 없음                       | void                    | 상세 화면 전환과 초기 조화를 요청한다.                   | 1.1.1에서 UI 상태를 전이하고 1.1.2에서 표시 데이터를 만든 뒤 1.1.3으로 렌더링한다.          |
| 1.1.1   | UIStateController -> UISTM                  | handle(event : UIEvent = SHOW_ALL_TRADING_DETAILS) : UITransitionResult                                   | 상세 화면 진입 event           | UI 전이 결과                | Main 화면에서 Trading Details 화면으로 상태를 전환한다. | UISTM은 화면 상태만 결정하며 조회 I/O는 수행하지 않는다. 반환 결과는 controller가 직접 사용한다. |
| 1.1.2   | UIStateController -> TradeHistoryController | getTradeDetails(period : HistoryPeriod = TODAY, side : TradeSide = ALL) : TradeDetailsResult              | 기본 기간 TODAY, 기본 side ALL | 상세 표시 결과                | 첫 진입에 필요한 거래 목록, ETH 보유량, 성과를 요청한다.      | 1.1.2.1~1.1.2.4의 결과를 하나의 TradeDetailsResult로 묶는다.                |
| 1.1.2.1 | TradeHistoryController -> TradeHistoryQuery | TradeHistoryQuery(startDate : LocalDate, endDate : LocalDate, side : TradeSide = ALL) : TradeHistoryQuery | 오늘 날짜 범위, 전체 side        | 조회 조건 객체                | 화면 기본 필터를 결합한 조회 조건을 만든다.                | HistoryPeriod.TODAY를 실제 시작일/종료일로 변환하고 side를 함께 보존한다.             |
| 1.1.2.2 | TradeHistoryController -> TradeHistory      | find(query : TradeHistoryQuery) : List<Trade>                                                             | 메시지 1.1.2.1의 query       | 조건에 맞는 Trade 목록         | 인메모리 거래 이력에서 조건에 맞는 체결을 찾는다.             | 조건과 일치하는 목록을 1.1.2의 TradeDetailsResult에 포함한다.                    |
| 1.1.2.3 | TradeHistoryController -> Account           | getHoldings(asset : String = "ETH") : Decimal                                                             | 자산 symbol                | 현재 ETH 보유 수량            | 상세 화면 요약 카드에 표시할 보유량을 조회한다.              | Account의 최신 REST/WebSocket 반영값을 반환한다. 별도 reply 메시지는 없다.          |
| 1.1.2.4 | TradeHistoryController -> Performance       | getPerformance() : Performance                                                                            | 없음                       | 현재 Performance snapshot | 수익률, 매도 성과, 실현손익 및 수수료를 조회한다.            | UI Rule의 당일 전체 수익률, 매도 성과, 당일 수수료 표시에 필요한 값을 제공한다.               |
| 1.1.3   | UIStateController -> TradeHistoryUI         | displayTradeDetails(details : TradeDetailsResult) : void                                                  | 결합된 상세 표시 결과             | void                    | 상세 화면을 렌더링한다.                            | 거래 테이블, 기간/side 기본 선택, ETH 보유량, 수익률, 매도 성과, 수수료를 한 번에 표시한다.      |

## 6.2 기간 및 매수·매도 필터 변경

| 번호  | 호출자 -> 수신자                          | Operation                                                                  | Parameter      | Return | 설명                                         | 동작 과정                                                             |
|-----|-------------------------------------|----------------------------------------------------------------------------|----------------|--------|--------------------------------------------|-------------------------------------------------------------------|
| 2   | User -> TradeHistoryUI              | selectFilter(period : HistoryPeriod, side : TradeSide) : void              | 선택 기간과 거래 side | void   | 사용자가 오늘/7일/30일/전체 기간 또는 전체/매수/매도 필터를 선택한다. | 두 필터의 현재 선택값을 결합해 2.1로 전달한다. 원본 수신자는 TradeHistoryUI이다.            |
| 2.1 | TradeHistoryUI -> UIStateController | tradeHistoryFilterChanged(period : HistoryPeriod, side : TradeSide) : void | 결합된 기간/side    | void   | 필터 변경 event를 controller                    | 원본 operation은 tradeHistoryFilterChanged()이다. 2.1.1에서 UI 선택 상태를 갱신 |

| 번호      | 호출자 -> 수신자                                  | Operation                                                                                           | Parameter             | Return          | 설명                           | 동작 과정                                                                                                                          |
|---------|---------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------|-----------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
|         |                                             |                                                                                                     |                       |                 | 에 전달한다.                      | 하고 2.1.1.2에서 데이터를 다시 조회한다.                                                                                                     |
| 2.1.1   | UIStateController -> UISTM                  | handle(event : UIEvent) : UITransitionResult                                                        | 선택된 필터를 담은 UI event   | UI 전이 결과        | 기간/side 필터의 UI 상태를 전이한다.     | 같은 그룹에서 하나의 선택만 활성화하고 조회 조건을 확정한다.                                                                                             |
| 2.1.2   | UIStateController -> TradeHistoryController | getTradeDetails(period : HistoryPeriod, side : TradeSide) : TradeDetailsResult                      | 현재 기간과 side           | 필터 결과           | 결합 조건에 맞는 거래 상세를 다시 요청한다.    | 2.1.2.1과 2.1.2.2의 조회 목록을 TradeDetailsResult의 trades로 반환하며, 호출자는 그 목록만 2.1.3에 전달한다. 기존 요약 정보의 불필요한 재조회는 새 메시지로 추가하지 않는다.        |
| 2.1.2.1 | TradeHistoryController -> TradeHistoryQuery | TradeHistoryQuery(startDate : LocalDate, endDate : LocalDate, side : TradeSide) : TradeHistoryQuery | 선택 기간의 실제 날짜 범위와 side | 조회 조건 객체        | 기간과 side를 하나의 조회 조건으로 만든다.   | TODAY, 최근 7일, 최근 30일, 전체 기간을 날짜 범위로 변환한다.                                                                                      |
| 2.1.2.2 | TradeHistoryController -> TradeHistory      | find(query : TradeHistoryQuery) : List<Trade>                                                       | 필터 query              | 조건에 맞는 Trade 목록 | 거래 이력에서 필터 조건과 일치하는 체결을 찾는다. | 결과가 없으면 빈 목록을 반환해 UI가 빈 상태를 표시한다.                                                                                              |
| 2.1.3   | UIStateController -> TradeHistoryUI         | displayFilteredTrades(trades : List<Trade>) : void                                                  | 2.1.2 반환 값의 trades    | void            | 필터링된 거래 행을 화면에 반영한다.         | UIStateController가 TradeDetailsResult.trades를 추출해 기존 테이블을 결과 목록으로 교체하고, 빈 목록이면 empty state를 표시한다. 원본 수신자 이름은 TradeHistoryUI이다. |

조회 결과, Account 보유량 및 Performance는 동기 operation의 반환값으로 결합할 수 있으므로 반환용 reply 메시지를 추가하지 않는다.

## 7. Case 4 - CSV Export

이 다이어그램의 순서는 다음과 같다.

- 1.1 계열: CSV 팝업 열기와 기본 option 생성
- 2.2 계열: 저장 위치 선택
- 3.3 계열: 기간, 날짜, 파일명 option 변경
- 4.4 계열: 검증, 거래 조회, CSV 생성, 성공/실패 표시

CSV PNG의 진입 Boundary 이름은 TradingHistoryUI이지만 History PNG의 TradeHistoryUI와 같은 거래 내역 상세 Boundary이므로 구현 클래스 목록에서는 TradeHistoryUI로 통합한다.

### 7.1 CSV 팝업 열기

| 번호    | 호출자 -> 수신자                            | Operation                                                                                                                                                        | Parameter       | Return       | 설명                                       | 동작 과정                                                            |
|-------|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|--------------|------------------------------------------|------------------------------------------------------------------|
| 1     | User -> TradeHistoryUI                | clickCSVExport() : void                                                                                                                                          | 없음              | void         | 사용자가 거래 내역 상세 화면의 CSV 내보내기 버튼을 누른다.      | 원본 수신자 이름은 TradingHistoryUI이다. Boundary가 1.1로 팝업 열기 event를 전달한다. |
| 1.1   | TradeHistoryUI -> UIStateController   | openCSVExport() : void                                                                                                                                           | 없음              | void         | CSV option 팝업을 열도록 요청한다.                 | 1.1.1에서 UI 상태를 전이하고, 1.1.2로 임시 option을 만든 뒤 1.1.3으로 표시한다.        |
| 1.1.1 | UIStateController -> UISTM            | handle(event : UIEvent = CSV_EXPORT_CLICKED) : UITransitionResult                                                                                                | CSV 팝업 열기 event | UI 전이 결과     | UI 상태를 CSV_EXPORT_POPUP_DISPLAYED로 전이한다. | UISTM은 팝업 표시 action만 결정하고 파일 I/O는 수행하지 않는다.                      |
| 1.1.2 | UIStateController -> CSVExportOptions | CSVExportOptions(period : CSVPeriod = TODAY, startDate : LocalDate = today, endDate : LocalDate = today, fileName : String = defaultFileName) : CSVExportOptions | 기본 기간, 날짜, 파일명  | 임시 option 객체 | 팝업에서 수정할 CSV option을 생성한다.               | 저장 위치는 아직 미선택이며, UI 문서의 기본 기간/기본 파일명 규칙을 적용한다.                   |

| 번호    | 호출자 -> 수신자                                    | Operation                                               | Parameter | Return | 설명                   | 동작 과정                                                     |
|-------|-----------------------------------------------|---------------------------------------------------------|-----------|--------|----------------------|-----------------------------------------------------------|
| 1.1.3 | UIStateController<br>-> csvPopup :<br>PopupUI | showCSVExportOptions(options : CSVExportOptions) : void | 기본 option | void   | CSV option 팝업을 표시한다. | 저장 위치, 기간 preset, 시작일/종료일, 파일명, 취소/내보내기 controls 를 렌더링한다. |

## 7.2 저장 위치 선택

| 번호      | 호출자 -> 수신자                                    | Operation                                                                   | Parameter       | Return             | 설명                         | 동작 과정                                                                                   |
|---------|-----------------------------------------------|-----------------------------------------------------------------------------|-----------------|--------------------|----------------------------|-----------------------------------------------------------------------------------------|
| 2       | User -> csvPopup :<br>PopupUI                 | selectSaveLocation() : void                                                 | 없음              | void               | 사용자가 위치 선택을 누른다.           | 노트에 따라 이 operation은 저장 경로 선택만 담당하며 기간/파일명은 변경하지 않는다.                                    |
| 2.1     | csvPopup : PopupUI<br>-> UIStateController    | saveLocationSelectionRequested() : void                                     | 없음              | void               | OS 디렉터리 선택 창 열기를 요청한다.     | 2.1.1에서 UI 상태를 바꾸고 2.1.2에서 picker를 실행한다.                                                |
| 2.1.1   | UIStateController<br>-> UISTM                 | handle(event : UIEvent = SAVE_LOCATION_SELECT_CLICKED) : UITransitionResult | 위치 선택 event     | UI 전이 결과           | 파일 browser open 상태를 처리한다.  | 선택 완료와 취소는 Path? 반환 값으로 구분한다.                                                           |
| 2.1.2   | UIStateController<br>-> CSVFileGateway        | chooseDirectory() : Path?                                                   | 없음              | 선택 경로 또는 취소 시 null | OS 디렉터리 선택을 Gateway에 위임한다. | 2.1.2.1의 반환 경로를 그대로 사용한다. 반환 화살표는 추가하지 않는다.                                             |
| 2.1.2.1 | CSVFileGateway -> File System                 | openDirectoryPicker() : Path?                                               | 없음              | 선택 경로 또는 null      | OS 폴더 선택 창을 연다.            | 사용자가 취소하면 null을 반환하고 기존 저장 경로를 유지한다.                                                    |
| 2.1.3   | UIStateController<br>-> CSVExportOptions      | setSaveLocation(path : Path) : void                                         | 선택된 경로          | void               | option의 저장 위치를 갱신한다.       | 원본 라벨은 괄호 없는 setSaveLocation이다. picker 취소 시 이 메시지를 실행하지 않는다.                            |
| 2.1.4   | UIStateController<br>-> csvPopup :<br>PopupUI | displaySaveLocation(path : Path) : void                                     | 선택된 non-null 경로 | void               | 선택 경로를 팝업에 표시한다.           | 2.1.2가 non-null Path를 반환한 경우에만 선택된 폴더 영역을 갱신한다. picker 취소 시 2.1.3과 함께 생략하고 기존 표시를 유지한다. |

## 7.3 기간, 날짜 및 파일명 변경

| 번호    | 호출자 -> 수신자                                 | Operation                                                                                                    | Parameter          | Return   | 설명                                     | 동작 과정                                                                                                                             |
|-------|--------------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------|----------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 3     | User -> csvPopup :<br>PopupUI              | changeCSVOptions(period : CSVPeriod, startDate : LocalDate?, endDate : LocalDate?, fileName : String) : void | 변경된 기간/날짜/파일명      | void     | 사용자가 내보낼 범위 또는 파일명을 변경한다.              | nullable 날짜는 이번 UI event에서 해당 날짜가 변경되지 않았음을 뜻한다. 다이어그램 노트에 따라 저장 경로는 이 operation의 책임이 아니다.                                        |
| 3.1   | csvPopup : PopupUI<br>-> UIStateController | csvOptionChanged(period : CSVPeriod, startDate : LocalDate?, endDate : LocalDate?, fileName : String) : void | 현재 입력값             | void     | option 변경 event를 controller에 전달한다.     | nullable 날짜는 기존 값을 유지하는 patch 의미다. 3.1.1에서 UI 상태/guard를 처리하고 3.1.2에서 임시 option에 적용한다.                                             |
| 3.1.1 | UIStateController -> UISTM                 | handle(event : UIEvent) : UITransitionResult                                                                 | 기간/날짜/파일명 변경 event | UI 전이 결과 | 기간 preset, 직접 날짜 선택 및 파일명 입력 상태를 전이한다. | 시작일이 종료일보다 늦은 경우 등 UI guard 결과를 반환한다.                                                                                             |
| 3.1.2 | UIStateController -> CSVExportOptions      | apply(period : CSVPeriod, startDate : LocalDate?, endDate : LocalDate?, fileName : String) : void            | 검증 가능한 현재 입력값      | void     | 임시 CSV option을 갱신한다.                   | TODAY는 오늘, WEEKLY는 오늘 포함 7일, MONTHLY는 오늘 포함 30일로 non-null 날짜를 맞춘다. CUSTOM에서는 전달된 날짜만 바꾸고 null인 날짜는 기존 non-null Attribute 값을 유지한다. |
| 3.1.3 | UIStateController -> csvPopup :<br>PopupUI | refresh(options : CSVExportOptions) : void                                                                   | 갱신된 option         | void     | 팝업 표시를 option과 동기화한다.                  | 선택된 preset, 날짜 field 활성화 상태, 파일명 preview와 validation 표시를 갱신한다.                                                                    |

## 7.4 검증, 조회 및 파일 생성

| 번호        | 호출자 -> 수신자                                             | Operation                                                                                                          | Parameter                                       | Return               | 설명                                              | 동작 과정                                                                                        |
|-----------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|----------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------|
| 4         | User -> csvPopup :<br>PopupUI                          | exportConfirmed() : void                                                                                           | 없음                                              | void                 | 사용자가 <b>내보내기</b> 를<br>확정한다.                     | PopupUI가 4.1로<br>재 option의 내보나<br>기를 요청한다.                                                   |
| 4.1       | csvPopup : PopupUI -><br>UIStateController             | exportCSV() : void                                                                                                 | 없음                                              | void                 | CSV 검증 및 생성을<br>시작한다.                           | 4.1.1 검증 결과에<br>라 실패면 4.1.3a<br>공이면 4.1.3b와<br>4.1.4로 진행한다                                   |
| 4.1.1     | UIStateController -><br>CSVExportOptions               | validate() :<br>ValidationResult                                                                                   | 없음                                              | 검증 결<br>과            | 저장 위치, 파일명, 기<br>간/날짜 범위를 검증<br>한다.             | 저장 위치 미선택, 빈<br>금지 문자를 가진 피<br>명, 시작일이 종료일<br>다 늦은 범위를 오류<br>반환한다.                           |
| 4.1.2     | UIStateController -><br>UISTM                          | handle(event : UIEvent) :<br>UITransitionResult                                                                    | validation 성공 또는 실패<br>event                    | UI 전이<br>결과          | 검증 결과에 맞는 UI<br>상태를 결정한다.                       | 실패면 popup 유저<br>성공이면 export 전<br>상태로 전이한다.                                                   |
| 4.1.3a    | UIStateController -><br>csvPopup : PopupUI             | showValidationError(result<br>: ValidationResult) : void                                                           | 실패 항목과 사유                                       | void                 | [실패 시] 유효하지<br>않은 입력을 표시한<br>다.                 | 잘못된 control을<br>하고 오류 원인을 표<br>한 뒤 이 branch를<br>표한다.                                         |
| 4.1.3b    | UIStateController -><br>csvPopup : PopupUI             | showExportInProgress() :<br>void                                                                                   | 없음                                              | void                 | [성공 시] CSV 생<br>성 중 상태를 표시한<br>다.               | 이후 4.1.4의<br>service 작업이 완<br>될 때까지 중복 내보<br>기를 막는다.                                         |
| 4.1.4     | UIStateController -><br>TradeHistoryController         | exportCSV(options :<br>CSVExportOptions) :<br>CSVExportResult                                                      | 검증된 option 객체                                   | 성공/실패<br>및 경<br>로/사유 | 실제 거래 조회와<br>CSV 생성을 요청한<br>다.                  | 노트대로 option 객<br>체를 인자로 전달하고<br>공 또는 실패 이유를<br>환반는다.<br>4.1.4.1~4.1.4.3<br>을 조정한다.           |
| 4.1.4.1   | TradeHistoryController<br>-> TradeHistoryQuery         | TradeHistoryQuery(startDate<br>: LocalDate, endDate :<br>LocalDate, side : TradeSide<br>= ALL) : TradeHistoryQuery | option에서 추출한 날짜 범위,<br>전체 side                  | 조회 조<br>건 객체         | CSVExportOptions<br>에서 거래 조회에 필요<br>한 조건을 추출한다. | CSV 그림에는 side<br>택이 없으므로 전체<br>수/매도를 대상으로<br>다.                                              |
| 4.1.4.2   | TradeHistoryController<br>-><br>TradeHistoryRepository | streamTrades(query :<br>TradeHistoryQuery) :<br>Stream<Trade>                                                      | CSV 조회 query                                    | Trade<br>stream      | 조건에 맞는 저장 이력<br>을 streaming 방식으<br>로 읽는다.       | 전체 파일을 메모리<br>복제하지 않고<br>4.1.4.2.1의 re<br>stream을 query로<br>터링한다.                            |
| 4.1.4.2.1 | TradeHistoryRepository<br>-> File System               | openReadStream(path : Path)<br>: InputStream                                                                       | 거래 이력 저장 경로                                     | 입력<br>stream         | 로컬 거래 이력 파일의<br>읽기 stream을 연다.                  | stream의 수명과<br>close 처리는<br>repository<br>operation 내부에<br>보장하며 별도 메시<br>지를 추가하지 않는다         |
| 4.1.4.3   | TradeHistoryController<br>-> CSVFileGateway            | writeCSV(trades :<br>Stream<Trade>, options :<br>CSVExportOptions) :<br>CSVExportResult                            | Trade stream, 출력 option                         | CSV 생<br>성 결과        | 조회 결과를 사용자가<br>지정한 CSV 파일로<br>쓴다.               | Gateway가 Trade<br>CSV header/row<br>Stream<String><br>으로 직렬화한 뒤<br>4.1.4.3.1에 파<br>생성을 위임한다. |
| 4.1.4.3.1 | CSVFileGateway -> File<br>System                       | createCustomizedCSV(path :<br>Path, fileName : String,<br>rows : Stream<String>) :<br>CSVExportResult              | 저장 위치, 파일명, 직렬화된<br>CSV 행 stream                | 생성 결<br>과            | 지정 경로에 직렬화된<br>CSV 데이터를 기록해<br>파일을 생성한다.        | 성공 시 생성 경로,<br>패 시 I/O 사유를 반<br>한다. 반환 reply 메<br>지는 그리지 않는다                                 |
| 4.1.5     | UIStateController -><br>UISTM                          | handle(event : UIEvent) :<br>UITransitionResult                                                                    | CSV_EXPORT_SUCCEEDED<br>또는<br>CSV_EXPORT_FAILED | UI 전이<br>결과          | service 결과에 따라<br>완료 또는 오류 상태<br>로 전이한다.        | 4.1.4의<br>CSVExportResult<br>event로 변환한다.                                                    |
| 4.1.6a    | UIStateController -><br>csvPopup : PopupUI             | showExportError(reason :<br>String) : void                                                                         | 실패 사유                                           | void                 | [실패 시] 내보내기<br>오류를 표시한다.                        | popup과 option을<br>지해 사용자가 수정                                                                 |

| 번호     | 호출자 -> 수신자                              | Operation                              | Parameter  | Return | 설명                       | 동작 과정                         |
|--------|-----------------------------------------|----------------------------------------|------------|--------|--------------------------|-------------------------------|
|        |                                         |                                        |            |        |                          | 다시 시도할 수 있다.                  |
| 4.1.6b | UIStateController -> csvPopup : PopupUI | showExportComplete(path : Path) : void | 생성된 CSV 경로 | void   | [성공 시] 내보내기 완료 상태를 표시한다. | 진행 표시를 제거하. 저장 결과를 사용자에게 알린다. |

## 8. 전체 클래스 Attribute 및 Operation

이 절은 네 다이어그램에 등장한 모든 시스템 클래스를 한 번씩 정리한다. `User`, `Binance REST API`, `Binance WebSocket`, `Local File System/File System`은 외부 Actor이므로 클래스 목록과 분리해 9절에 정리한다.

`AppShellUI`, `RecentOrderUI`, `TradeHistoryUI`, `PopupUI`는 2.3절에서 설명한 웹 UI용 `<<boundary>>` classifier이다. 아래 Attribute와 Operation은 논리적인 UI 상태와 호출 계약이며, CSS class나 ES class 사용을 요구하는 목록이 아니다.

Operation 목록은 다이어그램에서 실제로 수신하는 메시지와 Phase 0에서 안전상 필수로 구체화한 기존 책임을 기준으로 한다. 새 Operation이 필요한 경우에도 먼저 기존 27개 클래스의 책임을 확인한다. 이번에 추가한 주문 조회·취소·복구 Operation은 거래소 REST 정규화를 이미 소유한 `APIGateway`에 배치했으며 새 업무 클래스를 만들지 않았다.

### 8.1 AppShellUI

Stereotype: `<<boundary>>`

기능: 앱의 전역 UI Boundary로서 interface를 시작하고 REGIME 선택, 자동매매 시작 확인, 중지 확인을 받는다.

Attribute

- `uiStateController` : `UIStateController` {not null}
- `selectedRegime` : `RegimeType?` = null
- `tradingStatus` : `TradingStatus` = `STOPPED` {not null}

Operation

- `startInterface()` : void
- `selectRegime(regimeType : RegimeType)` : void
- `startConfirmed()` : void
- `stopConfirmed()` : void

### 8.2 UIStateController

기능: UI Boundary의 event를 UISTM과 업무 Controller에 전달하고, 선택된 UI action을 화면에 반영하는 facade이다.

Attribute

- `uiSTM` : `UISTM` {not null}
- `appShellUI` : `AppShellUI` {not null}
- `marketDataController` : `MarketDataController` {not null}
- `tradingController` : `TradingController` {not null}
- `tradeHistoryController` : `TradeHistoryController` {not null}
- `recentOrderUI` : `RecentOrderUI` {not null}
- `tradeHistoryUI` : `TradeHistoryUI` {not null}
- `csvPopup` : `PopupUI` {not null}
- `csvFileGateway` : `CSVFileGateway` {not null}
- `csvExportOptions` : `CSVExportOptions?` = null

Operation

- `selectRegime(regimeType : RegimeType)` : void
- `startTrading()` : void
- `stopTrading()` : void
- `liquidateRecoveredPosition()` : void
- `showAllTradingDetails()` : void
- `tradeHistoryFilterChanged(period : HistoryPeriod, side : TradeSide)` : void
- `openCSVExport()` : void
- `saveLocationSelectionRequested()` : void
- `csvOptionChanged(period : CSVPeriod, startDate : LocalDate?, endDate : LocalDate?, fileName : String)` : void
- `exportCSV()` : void

`UIStateController`의 위 Operation은 사용자 event를 받는 논리 facade이므로 void로 표현한다. Phase 7 구현에서는 `UiApplicationFacade`와 `BackendUiAdapter`가 이 event에 stable command ID와 최신 expected Context version을 결합한 뒤 메시지 6.1.1, 7.1.1, 8.1.1, 8R.1.1의 concrete typed application Operation을 호출한다.

### 8.3 UISTM

기능: UI Event-Action Table에 따라 화면, popup, filter 및 export 표시 상태를 결정한다.

Attribute

- `currentState : UIState {not null}`

#### Operation

- `run(initialEvent : UIEvent = APP_STARTED) : UITransitionResult`
- `handle(event : UIEvent) : UITransitionResult`

### 8.4 TradingController

기능: 계좌 로드, 선택 전략 연결, 자동매매 시작/중지 및 주문 실행을 조정한다. 거래 상태 결정은 TradingSTM에 위임한다.

#### Attribute

- `tradingSTM : TradingSTM {not null}`
- `context : TradingContext {not null}`
- `account : Account {not null}`
- `positionSnapshot : PositionSnapshot {not null}`
- `marketSnapshot : MarketSnapshot {not null}`
- `apiGateway : APIGateway {not null}`
- `websocketGateway : WebSocketGateway {not null}`
- `tradeHistoryController : TradeHistoryController {not null}`
- `riskPolicyState : RiskPolicyState {not null}`
- `manualKillActive : boolean = false {not null}`
- `manualKillCleanupComplete : boolean = true {not null}`
- `riskControlVersion : int = 0 {not null}`
- `selectedRegime : RegimeType? = null`
- `reconciliationCauseStatus : ReconciliationCauseStatus = MISSING {not null}`
- `reconciliationCauseCategory : ReconciliationCauseCategory? = null`

#### Operation

- `loadAccount(asset : String = "ETH") : Account`
- `fetchSelectedTradingLogic(regimeType : RegimeType) : TradingSTM`
- `commitRegimeSelection(regimeType : RegimeType, selectedSTM : TradingSTM?, commandId : String, expectedVersion : int) : TradingLogicSelectionResult`
- `updateSplitRatios(commandId : String, expectedVersion : int, scaleIn : Decimal, scaleOut : Decimal) : SplitRatioResult`
- `startTrading(commandId : String, expectedVersion : int) : TradingSessionResult`
- `stopTrading(commandId : String, expectedVersion : int) : TradingSessionResult`
- `liquidateRecoveredPosition(commandId : String, expectedVersion : int) : TradingSessionResult`
- `reconcileStartupState() : void`
- `reconnectAccountStreamAfterReconciliation(recoveryCommitObserver : Callable? = null) : Subscription`
- `observeOrderResult(result : OrderResult) : boolean`
- `observeMarketEvaluation(evaluation : MarketEvaluationSnapshot, marketVersion : int, sourceEventId : String) : TradingEvent?`
- `getReconciliationCauseSnapshot() : ReconciliationCauseSnapshot`
- `markEventRuntimeFailed() : void`
- `sealEventRuntimeFailureGate() : boolean`
- `markMarketStreamReconciliationRequired(reason : String) : void`
- `completeMarketStreamReconciliation(marketVersion : int) : void`
- `evaluateBuyRisk(order : Order) : RiskDecision`
- `setManualKill(active : boolean, commandId : String, expectedVersion : int) : ManualKillResult`
- `resumeManualKillCleanup() : boolean`
- `markProcessOwnershipAmbiguous(reason : ProcessOwnershipFailure) : void`

`fetchSelectedTradingLogic`은 `TYPE_0`을 정확히 109개 transition의 lower-BB registry에만 매핑하고 상단 BB 접촉은 `SAFE_TERMINATION`으로 완결한다. 다른 타입은 `UNSUPPORTED_TRADING_LOGIC`, active session의 변경은 `TRADING_ACTIVE`로 거부하며 기존 STM과 Context를 유지한다. selection은 start와 분리되어 있고, `fake` 또는 안전 gate를 모두 통과한 Phase 9 `testnet` mode와 명시적인 start command에서만 runtime을 시작한다. startup reconciliation은 durable history와 pending sidecar를 exchange open/recent/same-ID 사실에 대조한다. numeric `orderId`는 (`clientId`, `orderId`) pair로만 durable identity가 되며 terminal fill 집계도 Trade와 정확히 같아야 한다. history 저장과 sidecar REMOVE marker가 모두 해제되기 전에는 command를 허용하지 않는다. stream 재연결은 첫 account snapshot 뒤 새 signed stream ACK를 먼저 확보하고, 그 뒤 주문 snapshot/rebase와 두 번째 account snapshot을 완료한 조용한 barrier에서만 command를 다시 허용한다. 위 camelCase Operation은 Python의 `commit_regime_selection`, `update_split_ratios`, `start_trading`, `stop_trading`, `liquidate_recovered_position`, `observe_market_evaluation`, `set_manual_kill`, `resume_manual_kill_cleanup`에 각각 대응한다. `getReconciliationCauseSnapshot`은 Python의 `atomic_reconciliation_cause_snapshot` property, `markEventRuntimeFailed`는 `mark_event_runtime_failed`, `sealEventRuntimeFailureGate`는 `seal_event_runtime_failure_gate`에 대응한다. Snapshot은 같은 session lock에서 현재 `reconciliationRequired`, latch status와 category를 복사한다.

Category는 account stream unknown/external execution, prepare filter/cap reject, event worker/runtime failure, market stream failure, order/persistence ambiguity, process ownership ambiguity의 여섯 secret-free 값만 허용한다. 최초는 `EXACT`, 두 번째 같은 category는 `DUPLICATE`, 다른 category는 `CONFLICT`로 영구 잠그며, `MISSING`과 두 non-exact 상태는 category를 노출하지 않는다. 일반 account disconnect, app-prefix unknown, market/prepare/order origin은 current reconciliation bool이 복구 후 `false`가 돼도 historical latch가 남으므로 `false + EXACT + category`가 유효하다. Prefixless external execution, event worker/runtime과 process ownership은 process-lifetime blocker이므로 `false`와 함께 봉인할 수 없다. Event worker/runtime 실패는 terminal을 되살리지 않고 원인을 기록한 뒤 현재 session의 신규 effect gate를 fail closed한다. Failure finalizer 전용 seal은 terminal check와 active/reconciliation blocker commit을 같은 lock에서 수행하고, 기존 reconciliation cause를 다시 기록하지 않으면서 future reconnect를 맡는다. manual kill의 `expectedVersion`은 TradingContext가 아니라 별도 `riskControlVersion`이며 같은 command ID와 payload는 최초 결과를 재생한다. 최근 1,024개 이하 성공 command의 active/version, expectedVersion과 policy provenance는 toggle-no-op 모두 별도 strict JSONL v2에 fsync하므로 response loss 뒤 process가 재시작되어도 최초 결과와 ID/payload 충돌 판정을 복원한다. 복구 청산은 durable open lot의 REGIME과 owner를 검증한 liquidation-only session에

서만 `STOP_CONFIRMED`를 처리하며 `run()`이나 자동 전략 event를 실행하지 않는다. mutable `Position`이 authoritative 주문 체결 수량을 소유하고 `PositionSnapshot`은 Context publication과 start/stop Guard에 그 값을 전달한다.

확정된 manual kill behavior는 `CANCEL_AND_LIQUIDATE`다. Activation receipt를 먼저 fsync해 신규 BUY를 차단한 뒤 app-owned pending/UNKNOWN/open 주문을 same-ID query-cancel-reconcile하고, 잔여 Position을 기존 STOP/recovery path로 정확히 청산한다. 현재 구현은 durable pending same-ID query → 개별 cancel → same-ID terminal requery → partial History/Position reconcile → canonical STOP/recovery SELL 순서를 재사용한다. Restart는 control version에서 같은 cleanup identity를 재구성해 strategy `run()` 없이 재개하고, exact activation replay와 active no-op은 effect를 중복하거나 최초 activation의 behavior-policy version을 바꾸지 않는다. `manualKillCleanupComplete`는 account stream readiness와 app-owned open/pending/memory order, Context pending과 Position을 모두 권위 확인한 뒤에만 `true`다. RECON activation, reconnect same-ID re-cancel, partial residual SELL, fresh release TOCTOU와 cleanup-incomplete shutdown 차단의 focused local test는 통과했다. 실제 Phase 13 Testnet public-path와 local suite에 없는 추가 외부 timeout/5xx/persistence 조합은 GAP으로 유지한다.

Phase 13의 risk policy와 process ownership은 global command gate에 섞지 않는다. 신규 BUY만 `evaluateBuyRisk`를 반드시 통과하고, 안전 SELL, cancel, same-ID query, reconciliation, read-only 조회와 안전 종료는 risk 차단 중에도 허용한다. process owner, listener 또는 policy version이 불명확하면 신규 BUY를 차단하고 자동 kill이나 새 client-order ID 제출을 하지 않는다. Tauri는 launcher child handle/PID와 READY의 Python `runtime_pid/process_start_id`를 별도로 소유한다. Python의 `.backend-runtime.lock`은 그 runtime identity와 `ACTIVE|ORPHANED|RELEASED`만 durable하게 기록하며, port-parent PID-token을 기록하지 않는다. Parent control FD EOF는 `ORPHANED`를 fsync하고 order gate를 닫지만 listener/process/lock을 임의로 종료하지 않는다.

FD5 writer는 native parent만 소유하므로 parent가 종료되면 kernel EOF가 liveness 신호가 된다. Python은 시간 기반 heartbeat timeout을 추측하지 않고 EOF를 읽은 같은 waiter iteration에 ownership gate와 artifact fsync를 적용한다.

다음 native startup은 Python spawn 전에 ownership artifact의 exclusive lock, exact schema, owner-only regular single-link inode와 recorded PID 부재를 검증한다. stale `ACTIVE|ORPHANED`만 state/PID/start UUID를 operator dialog에 표시하며 명시적 확인 뒤 같은 device/inode-identity와 PID 부재를 다시 검증한 경우에만 같은 artifact를 `RELEASED`로 fsync하고 재시작한다. 취소, live 또는 permission-ambiguous PID, lock 경합, invalid artifact와 dialog 이후 교체는 mutation 없이 fail closed하며 process kill, order cancel-청산 또는 새 order submit을 수행하지 않는다.

## 8.5 TradingSTM

기능: trading event와 TradingContext를 사용해 자동매매 상태 및 수행 action을 결정한다.

### Attribute

- `currentState : TradingState {not null}`
- `regimeType : RegimeType {not null}`

### Operation

- `getSTMInstance(regimeType : RegimeType) : TradingSTM`
- `run(context : TradingContextView) : TradingSTMResult`
- `handle(event : TradingEvent, context : TradingContextView) : TradingSTMResult`
- `orderFinished(event : TradingEvent, context : TradingContextView) : TradingSTMResult`

`handle(event, context)`가 유일한 canonical decision API다. Python은 overload를 만들지 않는다. `orderFinished`는 허용된 concrete outcome만 검사한 뒤 `handle`로 위임한다.

## 8.6 TradingContext

기능: TradingSTM의 판정과 주문 실행에 필요한 runtime 값, 선택 REGIME, 분할 비율 및 현재 주문 의도를 보존한다.

### Attribute

- `account : Account {not null}`
- `position : PositionSnapshot {not null}`
- `selectedRegime : RegimeType {not null}`
- `lowerEventId : String? = null`
- `positionOwner : PositionOwner? = null`
- `pendingOrderSide : OrderSide? = null`
- `pendingStrategy : StrategyType? = null`
- `tradingPhase : TradingPhase = IDLE {not null}`
- `scaleInRatio : Decimal {not null}`
- `scaleOutRatio : Decimal {not null}`
- `version : Integer = 0 {not null}`

### Operation

- `initialize(account : Account, selectedRegime : RegimeType, position : PositionSnapshot, scaleInRatio : Decimal, scaleOutRatio : Decimal) : void`
- `applyTradingSTMResult(result : TradingSTMResult) : void`
- `getSplitRatio() : Decimal`

## 8.7 MarketDataController

기능: REST 과거 봉과 WebSocket buffer를 병합해 일관된 MarketSnapshot을 만들고, 같은 generation의 지속 Kline을 전략 평가 경계까지 전달한다.

### Attribute

- `apiGateway : APIGateway {not null}`
- `websocketGateway : WebSocketGateway {not null}`

- `marketSnapshot : MarketSnapshot {not null}`
- `regimeController : RegimeController {not null}`
- `symbol : String = "ETHUSD" {not null}`
- `intervals : Set<Interval> {not null}`

#### Operation

- `InitializeMarketData(symbol : String = "ETHUSD") : MarketSnapshot`
- `reconcileMarketStream() : MarketSnapshot`
- `observeKline(kline : Kline) : void`

30분 EMA는 period 9, 첫 9개 시간순 확정봉 종가의 SMA seed, alpha 0.2, 최근 EMA9 6개의  $x=0.5$  OLS, Decimal 유효숫자 34와 threshold 비교 전 최종 8자리 `ROUND_HALF_EVEN`으로 확정했다. 진행 30분봉의 realtime price, Case C `tp_price`와 확정 1분봉 `close_1m`은 같은 직전 확정 EMA에서 매번 독립 후보를 계산하고 확정 EMA 시 계열에 commit하지 않는다.

Raw OLS slope는 `raw_ols_slope / candidate_price * 100`, 단위  $\%/30$ 분봉으로 정규화한다. Actual close, realtime, Case C `tp_price`, 확정 1분봉 `close_1m` 계산은 각각 실제로 대입한 가격을 동일 계산의 분모로 사용한다. Shared Decimal helper와 concrete builder, monotonic tracker의 reset/rebase, immutable evaluation/version queue preparer와 production bootstrap observer wiring을 구현했다. Concrete builder는 기존 `MarketEvaluationBuilder` seam의 `MarketDataController` 소유 application 구현 세부사항이며, 신규 business lifeline 또는 package public API가 아니다. Architecture audit 대상에는 포함한다. Production builder golden-seed-threshold-HALF\_EVEN-non-accumulation-actual close-1분 결할 reset/rebase 직접 회귀 20개와 연속 market-version queue provenance를 통과했다. 4H 경계 12개와 UTC 자정 180개 유효 arrival permutation은 정확히 한 atomic version과 같은-version REGIME 평가를 검증한다. Public `observeKline`에서 시작하는 immediate/partial/UNKNOWN/failure/SELL/STOP 여섯 흐름과 production Spot REST memory-HTTP E2E도 private Action 호출 없이 통과했다. Actual Phase 13 Testnet order/fill trace는 이 local 완료와 별도로 남는다.

#### 8.8 APIGateway

기능: Binance REST API의 Kline, 계좌 및 주문 기능을 내부 타입으로 캡슐화한다.

#### Attribute

- `restClient : BinanceRESTClient {not null}`
- `symbol : String = "ETHUSD" {not null}`

#### Operation

- `loadAllKlines(symbol : String, limit : int) : Map<Interval, List<Kline>>`
- `fetchAccountSnapshot(asset : String = "ETH") : AccountSnapshot`
- `fetchCommissionDiscountPolicy(symbol : String) : CommissionDiscountPolicy`
- `fetchAccountAssetFilters(symbol : String) : List<AccountAssetFilter>`
- `fetchAccountRelevantFilters(symbol : String) : AccountRelevantFilters`
- `hasAnyExchangeOpenOrders() : Boolean`
- `hasAnyExchangeOpenOrderLists() : Boolean`
- `fetchReferencePrice(symbol : String) : ReferencePrice`
- `sellAllPosition(symbol : String, quantity : Decimal) : OrderResult`
- `submitOrder(order : Order) : OrderResult`
- `queryOrderResult(symbol : String, orderId : Long? = null, clientOrderId : String? = null) : OrderResult`
- `cancelOrder(symbol : String, orderId : Long? = null, clientOrderId : String? = null) : OrderResult`
- `listOpenOrderResults(symbol : String) : List<OrderResult>`
- `listRecentOrderResults(symbol : String, limit : int = 100) : List<OrderResult>`
- `listAllOpenOrderResults(symbol : String) : List<OrderResult>`
- `listAllRecentOrderResults(symbol : String, limit : int = 1000) : List<OrderResult>`

조회와 취소는 `orderId` 또는 `clientOrderId` 중 하나 이상을 요구한다. retry 횟수나 전략 판단은 이 Gateway가 아니라 `TradingController`가 ADR-002에 따라 수행한다.

Testnet 주문 준비는 signed `GET /api/v3/account/commission`의 `standardCommission`, `specialCommission`, `taxCommission` 네 비율과 discount flag를 엄격히 정규화한다. 제3 수수료 자산 가능성은 차단하며, dust 회계가 구현되기 전에는 MARKET BUY의 `taker + buyer` 합이 세 유형 중 하나라도 양수이면 수신 ETH 차감 가능성을 이유로 주문 전에 fail closed한다. 두 discount enable flag가 참이면 할인율 숫자가 0이어도 tax/special 수수료가 검증된 string discount asset으로 전환될 수 있으므로 제3 자산 가능성을 차단한다. 공식 endpoint schema는 `discountAsset` string을 요구하지만, 2026-08-24 실제 Spot Testnet은 두 flag가 참이고 discount와 12개 commission 비율이 모두 0인 응답에서 explicit `null`을 반환했다. 이 exact all-zero schema drift만 수수료 자산 부재로 허용하며, field 누락, 하나라도 양수인 null 또는 string이 아닌 다른 타입은 fail closed한다.

2026-08-31 재대조한 현재 공식 계약의 signed `GET /api/v3/myFilters`는 한 account와 symbol에 관련된 exchange/symbol/asset filter를 함께 반환한다. Gateway는 요청 symbol을 body와 별도의 provenance로 결속하고, 공식 union의 type-field-JSON type을 scope별 immutable `AccountRelevantFilters`로 엄격히 정규화한다.

Unknown/duplicate/extra field와 공개 평가식이 없는 `T_PLUS_SELL`은 무시하지 않고 fail closed한다. `fetchAccountAssetFilters`는 전체 payload를 같은 parser로 먼저 검증한 뒤 `MAX_ASSET` projection만 주는 호환 경계다. 공식 `MAX_ASSET` 정의는 base asset에는 quantity, quote asset에는 notional을 적용한다고만 명시하고 quantity 기반 MARKET 주문의 quote notional 환산 가격식은 제공하지 않는다. 따라서 이번 Phase 13의 quantity 기반 MARKET BUY/SELL에서는 base `MAX_ASSET`만 최종 제출 quantity로 평가하고, quote `MAX_ASSET`이 반환되면 `referencePrice * quantity`를 합성하지 않은 채 journal/POST 전에 고정 fail closed한다. 관련 없는 asset, 중복, unknown asset filter와 잘못된 Decimal도 주문 전에 fail closed한다. 별도 symbol filter인 MARKET `MIN_NOTIONAL/NOTIONAL`은 non-null reference price가 있으면 그 값을 사용한다. 따라서 fresh public `GET /api/v3/referencePrice`를 엄격한 `ReferencePrice`로 정규화하고 preflight와 각 submit-time evidence에 각각 관찰 시각을 결속한다. Null 또는 `-2043`, symbol-timestamp drift에서는 임의 VWAP-마지막 가격을 추정하지 않는다. `MAX_POSITION`은 current account exposure와 모든 open BUY까지 필요한 filter이므로 이 좁은 Phase 13 BUY target에서는 존재 자체를 차단하고, 정확한 account-position evaluator가 없는 상태에서 quantity만 비교해 통과시키지 않는다.

`EXCHANGE_MAX_NUM_*`와 symbol count filter는 symbol-scoped order 조회로 account 전체를 추정하지 않는다. Preflight와 각 submit-time prepare는 signed `GET /api/v3/openOrders`를 symbol 없이, signed `GET /api/v3/openOrderList`를 전용 endpoint로 조회해 두 exact empty snapshot과 각각의 관찰 시각을 고정한다. Non-empty/malformed/시각 순서 drift는 journal과 `POST /api/v3/order` 전에 차단하고 order/list ID를 Gateway-trace-오류 출력으로 노출하지 않는다.

Prepared composite evidence는 account filter, public rule, 두 account-wide empty-state와 reference price 중 가장 이른 관찰부터 고정 30초까지만 유효하다. 정확히 30초는 허용하지만 30초+1ms와 마지막 관찰보다 후퇴한 clock은 fingerprint를 한 번 소모한 뒤에도 order POST 전에 차단한다. 실제 transport는 socket I/O 직전 `before_send` guard에서 같은 server-aligned 시각으로 freshness를 다시 검증하고, 통과한 경우에만 최초 submission-attempt evidence를 만든다. 따라서 fingerprint 소비 뒤 transport 진입이 지연돼도 attempt evidence와 order POST 없이 fail closed한다.

Phase 13 trace v3는 signed composite와 별도로 같은 `exchangeInfo`에서 만든 `public_relevant_filters` exact projection을 보존한다. Validator는 quantity·notional·count·passive·`MAX_POSITION`의 signed/public overlap, public scalar rule 결속과 위 30초 인과관계를 runtime과 같은 규칙으로 재검증한다. Stale-clock regression-schema drift는 차단하고 preserved trace v2는 그 version의 계약으로만 계속 검증한다.

Phase 13 actual preflight의 격리 조치는 application prefix에 한정하지 않고 manual client ID까지 포함한 symbol-scoped `openOrders`와 `allOrders(limit=1000)`를 사용한다. Open 결과는 비어 있어야 하고 recent 결과의 (`exchange order ID`, `client order ID`) exact set은 inode-digest-pending-Position 검증을 통과한 closed baseline Trade set과 같아야 한다. Fresh baseline이면 recent도 비어 있어야 한다. Missing, duplicate, manual/외부 identity 또는 stable exchange ID 부재는 actual mutation 전에 고정 문장으로 fail closed하며, 실패 출력에 `OrderResult/Trade repr`, ID와 fill을 반사하지 않는다.

## 8.9 WebSocketGateway

기능: Binance WebSocket 구독을 만들고 Kline 및 account event를 내부 형식으로 정규화한다.

### Attribute

- `websocketClient` : `BinanceWebSocketClient` {not null}
- `klineBuffer` : `Map<Interval, List<Kline>>` = {} {not null}
- `klineSubscription` : `Subscription?` = null
- `accountSubscription` : `Subscription?` = null
- `accountReady` : `boolean` = false {not null}

### Operation

- `startAllKlineBuffering(symbol : String, intervals : Set<Interval>) : Subscription`
- `promoteKlineBufferToLive(observer : Callable<Kline>, subscription : Subscription? = null) : Subscription`
- `startAccountInfoStream() : Subscription`
- `rebaseOrderResults(results : List<OrderResult>) : void`
- `isAccountReady() : boolean`

`accountReady`는 현재 세대의 transport가 연결되어 있고 bounded FIFO account dispatcher에 queued 또는 실행 중인 callback이 없을 때만 참이다. REST reconciliation 결과를 `rebaseOrderResults(...)`로 먼저 심은 뒤 queued stream event를 단조·역등 병합한다.

## 8.10 MarketSnapshot

기능: 동일 평가 시점의 시간대별 Kline과 현재 ETH 가격을 보존한다.

### Attribute

- `symbol` : `String` = "ETHUSDT" {not null}
- `klinesByInterval` : `Map<Interval, List<Kline>>` = {} {not null}
- `currentETHPrice` : `Decimal` {not null}
- `updatedAt` : `Instant` {not null}

### Operation

- `update(klines : Map<Interval, List<Kline>>) : void`
- `getCurrentETHPrice() : Decimal`

## 8.11 RegimeController

기능: MarketSnapshot에서 같은 version의 4H 판정 입력을 만들고 RegimeSTM의 두 microstep 결과와 Action을 수행하며, 추천 REGIME과 사용자 선택 REGIME을 분리해 보존한다.

### Attribute

- `marketSnapshot` : `MarketSnapshot` {not null}
- `indicatorSnapshot` : `IndicatorSnapshot` {not null}
- `regimeSTM` : `RegimeSTM` {not null}
- `tradingController` : `TradingController` {not null}
- `recommendedRegime` : `RegimeType?` = null
- `selectedRegime` : `RegimeType?` = null

### Operation

- `calculate4HIndicators(snapshot : MarketSnapshot) : IndicatorSnapshot`
- `recommendRegime(indicators : IndicatorSnapshot) : RegimeType`
- `reconcileRegime(snapshot : MarketSnapshot) : RegimeResult?`
- `setRegimeType(regimeType : RegimeType, commandId : String, expectedVersion : int) : TradingLogicSelectionResult`

`recommendRegime`은 호환 facade이며 내부에서 `RegimeSTM.handle(...)`와 Action dispatcher를 사용한다. `setRegimeType`만 `selectedRegime`을 바꿀 수 있고 추천 Action에서는 호출하지 않는다. Phase 7 Python Operation `set_regime_type`은 `command_id`와 `expected_version`을 keyword-only 인자로 받고 `TradingLogicSelectionResult`를 반환한다.

## 8.12 IndicatorSnapshot

기능: 동일 평가 시점의 REGIME 판정용 4H 지표를 보존한다.

### Attribute

- timeframe : Interval = FOUR\_HOURS {not null}
- ema9Series : List<Decimal> = [] {not null}
- ema9Slope : Decimal {not null}
- swingStructure : SwingStructure {not null}
- liveEma9 : Decimal {not null}
- calculatedAt : Instant {not null}

### Operation

- update(ema9Series : List<Decimal>, ema9Slope : Decimal, swingStructure : SwingStructure, liveEma9 : Decimal) : void

## 8.13 RegimeSTM

기능: 4H 평가 event와 불변 Context로 guard, 다음 상태와 typed Action 요청을 결정한다. 지표 계산, Controller 상태 변경과 외부 I/O는 수행하지 않는다.

### Attribute

- currentState : RegimeState {not null}

### Operation

- handle(event : RegimeEvent, context : RegimeEvaluationContext? = null) : RegimeSTMResult

## 8.14 Order

기능: 실제 거래소 주문 전 로컬 주문 의도와 Binance 주문/fill 결과를 하나의 aggregate로 보존한다.

### Attribute

- intentId : String {not null}
- clientOrderId : String {not null}
- submissionAttempt : int = 0 {not null}
- riskPolicyVersion : int? = null
- symbol : String {not null}
- side : OrderSide {not null}
- strategy : StrategyType {not null}
- requestedQuantity : Decimal {not null}
- marketPriceAtDecision : Decimal {not null}
- exchangeOrderId : Long? = null
- status : OrderStatus? = null
- filledQuantity : Decimal = 0 {not null}
- filledAmount : Decimal = 0 {not null}
- fillPrice : Decimal? = null
- processedAt : Instant? = null
- fills : List<Fill> = [] {not null}
- failureReason : String? = null

### Operation

- Order(symbol : String, side : OrderSide, strategy : StrategyType, requestedQuantity : Decimal, marketPriceAtDecision : Decimal) : Order
- applyOrderResult(result : OrderResult) : void
- reapplyOrderResult(result : OrderResult) : void
- buildExecutionSummary() : ExecutionSummary

## 8.15 Position

기능: 현재 전략 소유 포지션의 수량, 평균 진입가, 취득원가 및 상태를 보존한다.

### Attribute

- symbol : String = "ETHUSD" {not null}
- owner : PositionOwner? = null
- quantity : Decimal = 0 {not null}
- averageEntryPrice : Decimal = 0 {not null}
- costBasis : Decimal = 0 {not null}
- enteredAt : Instant? = null
- status : PositionStatus = CLOSED {not null}
- exitReason : ExitReason? = null

### Operation

- `getCostBasis(executedQuantity : Decimal) : Decimal`
- `applyExecution(summary : ExecutionSummary) : void`

### 8.16 Trade

기능: 실제 체결 결과를 조회와 영속화에 적합한 단일 거래 기록으로 보존한다.

#### Attribute

- `tradeId : String {not null}`
- `orderId : Long {not null}`
- `clientOrderId : String {not null}`
- `executedAt : Instant {not null}`
- `side : OrderSide {not null}`
- `regimeType : RegimeType {not null}`
- `fillPrice : Decimal {not null}`
- `requestedQuantity : Decimal {not null}`
- `quantity : Decimal {not null}`
- `amount : Decimal {not null}`
- `feeAmount : Decimal {not null}`
- `feeAsset : String {not null}`
- `feeQuoteAmount : Decimal {not null}`
- `strategy : StrategyType {not null}`
- `marketPriceAtDecision : Decimal {not null}`
- `allocatedCostBasis : Decimal? = null`
- `realizedPnl : Decimal? = null`
- `realizedReturnRate : Decimal? = null`
- `exitReason : ExitReason? = null`

#### Operation

- `Trade(order : Order, summary : ExecutionSummary, realizedResult : RealizedResult?) : Trade`

### 8.17 TradeHistory

기능: 인메모리 Trade 목록을 보관하고 조건 조회의 원천이 된다.

#### Attribute

- `trades : List<Trade> = [] {not null}`

#### Operation

- `TradeHistory(trades : List<Trade>) : TradeHistory`
- `addTrade(trade : Trade) : void`
- `find(query : TradeHistoryQuery) : List<Trade>`

### 8.18 Performance

기능: 거래 이력으로부터 KST 당일/누적 realized 수익률, fee 포함 실현손익, 수수료 및 매도 성과를 ADR-004 공식으로 계산하고 보존한다.

#### Attribute

- `dailyReturnRate : Decimal = 0 {not null}`
- `cumulativeReturnRate : Decimal = 0 {not null}`
- `realizedPnl : Decimal = 0 {not null}`
- `dailyFee : Decimal = 0 {not null}`
- `totalFee : Decimal = 0 {not null}`
- `averageSellReturnRate : Decimal = 0 {not null}`
- `totalProfit : Decimal = 0 {not null}`
- `winningSellCount : int = 0 {not null}`
- `losingSellCount : int = 0 {not null}`

#### Operation

- `Performance(trades : List<Trade>) : Performance`
- `calculateRealizedResult(summary : ExecutionSummary, costBasis : Decimal) : RealizedResult`
- `applyNewTrade(trade : Trade) : void`
- `getPerformance() : Performance`

### 8.19 TradeHistoryController

기능: 거래 저장, 상세 조회, 성과 계산, manual kill receipt, repository 접근 및 CSV 내보내기를 조정한다.

#### Attribute

- `tradeHistory : TradeHistory {not null}`

- performance : Performance {not null}
- account : Account {not null}
- repository : TradeHistoryRepository {not null}
- csvFileGateway : CSVFileGateway {not null}

#### Operation

- loadTradeHistory() : TradeHistory
- recordOrderExecution(order : Order, summary : ExecutionSummary, costBasis : Decimal?) : void
- getTradeDetails(period : HistoryPeriod = TODAY, side : TradeSide = ALL) : TradeDetailsResult
- exportCSV(options : CSVExportOptions) : CSVExportResult
- getManualKillControlState() : ManualKillControlState
- getManualKillControlReplay() : List<ManualKillControlState>
- saveManualKillControlState(state : ManualKillControlState) : void

### 8.20 TradeHistoryRepository

기능: 로컬 Trade의 전체 읽기·단건 저장·streaming 조회와 manual kill receipt 복구를 담당한다.

#### Attribute

- storagePath : Path {not null}
- manualKillControlStoragePath : Path {not null}
- fileSystem : FileSystem {not null}

#### Operation

- getTradeHistory() : List<Trade>
- saveThisTradeByOrderID(orderId : Long, trade : Trade) : void
- streamTrades(query : TradeHistoryQuery) : Stream<Trade>
- savePendingOrder(order : Order) : void
- transitionPendingOrderLifecycle(clientOrderId : String, lifecycle : PendingOrderRecoveryLifecycle) : void
- removePendingOrder(clientOrderId : String) : void
- getManualKillControlState() : ManualKillControlState
- getManualKillControlReplay() : List<ManualKillControlState>
- saveManualKillControlState(state : ManualKillControlState) : void

Pending order sidecar는 **PREPARED**, **SUBMITTED**, **PARTIAL**, **UNKNOWN**, **TERMINAL**, **HISTORY\_COMMITTED**, **SUBMISSION\_REJECTED\_CONFIRMED** 전이를 append-only로 fsync한다. 제출 시도 번호와 policy version은 UPSERT 전에 예약되어 restart 뒤에도 같은 intent의 총 제출 예산과 risk provenance가 복원된다. timeout 또는 decode failure는 실패가 아니라 **UNKNOWN**이며 새 identity를 만들지 않는다.

Schema v3 writer는 REST POST 직전 **SUBMITTED**를 fsync하며 전이 실패 시 POST를 호출하지 않는다. 따라서 v3 **PREPARED**는 POST 미시작 provenance로 replay하고, 1-2-4-8초 same-ID 조회가 모두 exact absence인 경우만 REMOVE한다. Legacy v1/v2 **PREPARED**는 POST 직후 crash를 배제할 수 없으므로 계속 fail closed하며, 동일 ID의 후속 UPSERT로 provenance를 업그레이드하지 않는다. REMOVE는 active lock만 해제하고 기존 intent/attempt audit line을 지우지 않는다.

Manual kill control journal은 pending journal과 별도 파일이다. Schema v1의 연속 toggle을 호환 replay하고 schema v2는 toggle과 no-op 성공 command를 모두 기록한다. 각 v2 line의 **expectedVersion**은 직전 authoritative version과 같아야 하고, active가 바뀌면 result version이 1 증가하며 no-op이면 그대로다. 최근 1,024개 이하 receipt를 일반 UI command eviction과 독립된 restart 전용 command cache에 복원해 같은 ID-payload는 최초 결과를 재생하고 다른 payload는 거부한다. 매 load/append는 same-inode single-link regular file의 strict replay, cached receipt와 stat anchor 및 post-write replay를 결합한다. 파일 부재만 최초 inactive version 0이며, 실행 중 empty-unlink-valid-prefix rollback, path 교체, partial tail-중복 key-추가 field와 version/ID 충돌은 fail closed한다. Cached receipt가 있는 shutdown barrier는 missing path도 검증해 정상 종료 ACK를 내보내지 않는다.

### 8.21 Account

기능: 자산별 잔액, ETH 보유량, 현재가와 평가금액을 보존한다.

#### Attribute

- balances : Map<String, Decimal> = {} {not null}
- currentPrice : Decimal = 0 {not null}
- valuation : Decimal = 0 {not null}
- updatedAt : Instant {not null}

#### Operation

- getHoldings(asset : String = "ETH") : Decimal

### 8.22 RecentOrderUI

Stereotype: <<boundary>>

기능: 메인 화면의 최근 체결 목록과 거래 상세 화면 진입점을 제공한다.

#### Attribute

- uiStateController : UIStateController {not null}
- recentTrades : List<Trade> = [] {not null}

## Operation

- `showAllTradingDetails() : void`

## 8.23 TradeHistoryUI

Stereotype: <<boundary>>

기능: 거래 내역 상세, 요약, 기간/side filter 및 CSV 내보내기 진입점을 표시한다.

## Attribute

- `uiStateController : UIStateController {not null}`
- `selectedPeriod : HistoryPeriod = TODAY {not null}`
- `selectedSide : TradeSide = ALL {not null}`
- `displayedTrades : List<Trade> = [] {not null}`
- `details : TradeDetailsResult? = null`

## Operation

- `selectFilter(period : HistoryPeriod, side : TradeSide) : void`
- `displayTradeDetails(details : TradeDetailsResult) : void`
- `displayFilteredTrades(trades : List<Trade>) : void`
- `clickCSVExport() : void`

## 8.24 TradeHistoryQuery

기능: 기간과 거래 side를 결합한 불변 조회 조건을 표현한다.

## Attribute

- `startDate : LocalDate {not null}`
- `endDate : LocalDate {not null}`
- `side : TradeSide = ALL {not null}`

## Operation

- `TradeHistoryQuery(startDate : LocalDate, endDate : LocalDate, side : TradeSide = ALL) : TradeHistoryQuery`

## 8.25 PopupUI

Stereotype: <<boundary>>

기능: 역할명 `csvPopup`으로 CSV option 입력, validation 오류, 진행 상태 및 완료/실패를 표시한다.

## Attribute

- `options : CSVExportOptions {not null}`
- `isVisible : bool = false {not null}`
- `validationError : String? = null`
- `exportStatus : CSVExportStatus = IDLE {not null}`

## Operation

- `selectSaveLocation() : void`
- `changeCSVOptions(period : CSVPeriod, startDate : LocalDate?, endDate : LocalDate?, fileName : String) : void`
- `exportConfirmed() : void`
- `showCSVExportOptions(options : CSVExportOptions) : void`
- `displaySaveLocation(path : Path) : void`
- `refresh(options : CSVExportOptions) : void`
- `showValidationError(result : ValidationResult) : void`
- `showExportInProgress() : void`
- `showExportError(reason : String) : void`
- `showExportComplete(path : Path) : void`

## 8.26 CSVExportOptions

기능: CSV 저장 위치, 기간, 시작일/종료일 및 파일명을 하나의 임시 option 객체로 보존하고 검증한다.

## Attribute

- `saveLocation : Path? = null`
- `period : CSVPeriod = TODAY {not null}`
- `startDate : LocalDate {not null}`
- `endDate : LocalDate {not null}`
- `fileName : String {not null}`

## Operation

- CSVExportOptions(period : CSVPeriod = TODAY, startDate : LocalDate = today, endDate : LocalDate = today, fileName : String = defaultFileName) : CSVExportOptions
- setSaveLocation(path : Path) : void
- apply(period : CSVPeriod, startDate : LocalDate?, endDate : LocalDate?, fileName : String) : void
- validate() : ValidationResult

## 8.27 CSVFileGateway

기능: OS directory picker와 CSV 파일 쓰기를 UI/Controller에서 분리한다.

Attribute

- fileSystem : FileSystem {not null}

Operation

- chooseDirectory() : Path?
- writeCSV(trades : Stream<Trade>, options : CSVExportOptions) : CSVExportResult

## 8.28 RiskPolicy

Stereotype: <<value object>>

기능: 사용자가 승인한 단건·누적 position·KST daily loss 및 manual kill 동작을 한 version으로 묶는다. Configured policy의 세 상한에서 null/언어별 None은 명시적 무제한을 뜻한다. Policy 문서·주입 자체가 없거나 strict conversion에 실패한 UNAVAILABLE과 구분하며, None을 숫자 0이나 거대한 Decimal로 치환하지 않는다.

Attribute

- version : int {not null}
- maxOrderNotional : Decimal? = null
- maxPositionNotional : Decimal? = null
- maxDailyLoss : Decimal? = null
- dailyLossScope : DailyLossScope {not null}
- manualKillBehavior : ManualKillBehavior {not null}

2026-08-29 configured 값은 세 상한 모두 null, REALIZED\_ONLY, CANCEL\_AND\_LIQUIDATE다. 상한이 null이어도 후보 주문·현재·예약·예상 Position notional과 KST 실현 손실을 Decimal로 계산·게시하며 해당 비교만 건너뛰다. 유한 Decimal이면 기존 차단 우선순위와 equality 경계를 유지한다.

## 8.29 RiskBudgetSnapshot

Stereotype: <<value object>>

기능: 한 BUY 판단에서 사용한 authoritative source version과 현재·예약·후보 노출 및 KST 손실을 불변으로 보존한다.

Attribute

- policyVersion : int? {not null when configured}
- marketVersion : int {not null}
- accountVersion : int {not null}
- contextVersion : int {not null}
- currentPositionNotional : Decimal {not null}
- reservedBuyNotional : Decimal {not null}
- candidateOrderNotional : Decimal {not null}
- projectedPositionNotional : Decimal {not null}
- dailyRealizedPnl : Decimal {not null}
- unrealizedPnl : Decimal {not null}
- dailyLoss : Decimal {not null}
- manualKillActive : boolean {not null}

## 8.30 RiskDecision

Stereotype: <<value object>>

기능: 신규 BUY 허용 여부와 첫 typed 차단 사유를 해당 판단의 위험 예산에 연결한다.

Attribute

- allowed : boolean {not null}
- budget : RiskBudgetSnapshot {not null}
- blockReason : RiskBlockReason? = null

허용 결과에는 차단 사유가 없고 차단 결과에는 정확히 하나의 typed 사유가 있다. 차단 우선순위는 policy availability/version, manual kill, 단건 notional, KST daily loss, 누적 position notional 순서이며 같은 입력 snapshot은 같은 첫 사유를 반환한다. Configured null 상한은 그 비교만 건너뛰지 policy availability/version 또는 manual kill 차단을 건너뛰지 않는다.

## 8.31 ManualKillResult

Stereotype: <<value object>>

기능: manual kill command의 활성 상태와 policy-control version provenance를 불변 보존한다.

#### Attribute

- `active` : `boolean {not null}`
- `behavior` : `ManualKillBehavior?`
- `policyVersion` : `int?`
- `riskControlVersion` : `int {not null}`

동일 command ID와 payload는 최초 결과를 재생하며 다른 payload 재사용과 stale `riskControlVersion`은 mutation 없이 거부한다. policy가 `UNAVAILABLE`이어도 manual kill 활성화는 허용해 신규 BUY fail-closed 상태를 강화할 수 있다.

### 8.32 ManualKillControlState

Stereotype: `<<value object>>`

기능: 한 manual kill 성공 command receipt와 그 결과의 authoritative active/control version 및 policy provenance를 restart-safe 값으로 보존한다.

#### Attribute

- `active` : `boolean {not null}`
- `version` : `int {not null}`
- `commandId` : `String?`
- `expectedVersion` : `int?`
- `behavior` : `ManualKillBehavior?`
- `policyVersion` : `int?`

Command가 없는 inactive version 0만 파일 부재의 초기 상태다. Schema v2 receipt는 command ID와 expected version을 반드시 가지며, toggle 결과는 version을 1 증가시키고 no-op 결과는 같은 version을 유지한다. behavior와 policy version은 configured policy에서만 함께 존재한다.

이 다섯 타입은 기존 27개 business lifeline에 coordinating class를 추가하지 않는 immutable value object다. 계산과 정책 적용 Operation의 owner는 계속 `TradingController`다.

## 9. 외부 Actor의 호출 계약

외부 Actor는 시스템 클래스의 Attribute/Operation 목록 대상은 아니지만, 다이어그램에서 호출되는 계약을 구현 참고용으로 정리한다.

### 9.1 Binance REST API

- `get1mKlines(symbol : String, limit : int) : List<Kline>`
- `get30mKlines(symbol : String, limit : int) : List<Kline>`
- `get4hKlines(symbol : String, limit : int) : List<Kline>`
- `get1dKlines(symbol : String, limit : int) : List<Kline>`
- `getAccount() : BinanceAccountResponse`
- `getAccountCommission(symbol : String) : BinanceCommissionResponse`
- `placeOrder(symbol : String, side : OrderSide, quantity : Decimal) : BinanceOrderResponse`
- `getOrderStatus(symbol : String, orderId : Long) : BinanceOrderStatusResponse`
- `getAccountTrades(symbol : String, orderId : Long) : List<Fill>`
- `getOrderByClientOrderId(symbol : String, clientOrderId : String) : BinanceOrderStatusResponse`
- `cancelOrder(symbol : String, orderId : Long?, clientOrderId : String?) : BinanceOrderResponse`
- `getOpenOrders(symbol : String) : List<BinanceOrderResponse>`
- `getRecentOrders(symbol : String, limit : int) : List<BinanceOrderResponse>`
- `sellAllPosition(symbol : String, quantity : Decimal) : BinanceOrderResponse`

### 9.2 Binance WebSocket

- `subscribeAllKlineStreams(symbol : String, intervals : Set<Interval>) : Subscription`
- `subscribeAccountInfo() : Subscription`

### 9.3 Local File System / File System

- `read(path : Path) : String`
- `write(path : Path, data : String) : void`
- `openDirectoryPicker() : Path?`
- `openReadStream(path : Path) : InputStream`
- `createCustomizedCSV(path : Path, fileName : String, rows : Stream<String>) : CSVExportResult`

## 10. 확정 정책과 Operation 추적성

### 10.1 적용 ADR

| ADR                                                             | 참고 정책                                                                                                                                                |
|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">ADR-001-canonical-regime-and-trading-mapping.md</a> | canonical REGIME/wire 값, <code>TYPE_0</code> -> <code>LOWER_BB</code> 109개와 상단 <code>SAFE_TERMINATION</code> , 미지원 거부, 두 STM signature, active 변경 금지 |

| ADR                                                | 참고 정책                                                                                    |
|----------------------------------------------------|------------------------------------------------------------------------------------------|
| ADR-002-order-retry-and-reconciliation.md          | timeout/partial/unknown/cancel/restart, retry 횟수와 순서                                     |
| ADR-003-stop-and-product-mode.md                   | Spot ETHUSDT long-only, stop guard, 실행 mode와 live gate                                   |
| ADR-004-persistence-performance-and-csv.md         | 4H 지표 golden vector, JSONL, Performance/KST, summary, CSV                                |
| ADR-005-loopback-transport-and-sidecar-security.md | endpoint, 응답/event envelope, token, sequence와 reconnect                                  |
| ADR-006-phase13-risk-recovery-and-readiness.md     | versioned risk gate, durable intent, process ownership, market resync, evidence와 live 잠금 |

ADR의 세부 numeric/schema 표는 이 문서의 Operation이 구현할 정책이다. 서로 충돌하면 더 구체적인 ADR을 먼저 적용하고 본 문서와 roadmap을 같은 변경 묶음에서 동기화한다.

10.2 REGIME 지원과 상품 범위

| Domain | Wire  | Phase 6 trading registry                          | Registry start Guard      |
|--------|-------|---------------------------------------------------|---------------------------|
| TYPE_0 | type0 | LOWER_BB 정확히 109개 transition, 상단 SAFE_TERMINATION | READY                     |
| TYPE_1 | type1 | 없음                                                | UNSUPPORTED_TRADING_LOGIC |
| TYPE_2 | type2 | 없음                                                | UNSUPPORTED_TRADING_LOGIC |
| TYPE_3 | type3 | 없음                                                | UNSUPPORTED_TRADING_LOGIC |
| TYPE_4 | type4 | 없음                                                | UNSUPPORTED_TRADING_LOGIC |

TYPE\_0의 UpperBandPolicy.SAFE\_TERMINATION은 새 상단 매매 전략이 아니다. G-07에서 pending 주문이 있으면 포지션 유무와 관계없이 STOPPING으로 전이해 UPPER\_BAND\_SAFE\_TERMINATION으로 취소하고 같은 ID를 stop\_after\_reconciliation = True로 조정하며 즉시 전량 매도하지 않는다. pending이 없고 포지션이 있으면 STOPPING에서 전략 평가를 취소하고 전량 매도한 뒤 G-06F/G-06R로 완결한다. 둘 다 없으면 lower event, Case B/C Context와 pending 필드를 정리하고 TERMINATED를 적용한 뒤 session 평가와 runtime을 즉시 종료한다.

상품은 Binance Spot ETHUSDT long-only다. Margin/Futures/short와 naked sell은 금지한다. TYPE\_0의 registry coverage와 session orchestration이 준비되어도 command\_enabled는 실행 mode gate와 분리한다. fake는 true이고 Phase 9 testnet은 고정 endpoint, startup reconciliation, 별도 주문 opt-in과 양수 BUY entry max-notional 상한뿐 아니라 현재 account stream의 connected-caught-up barrier를 모두 만족할 때만 true다. disabled, read-only testnet과 live는 false다. TYPE\_1~TYPE\_4는 추천·표시·선택을 허용하되 start만 차단한다. 미지원 타입을 TYPE\_0이나 lower-BB로 대체하지 않는다. 실행 중 REGIME 변경은 TRADING\_ACTIVE로 거부하고 stop 완료 뒤 새 session을 요구한다.

10.3 실행 모드

ExecutionMode는 disabled, fake, testnet, live 네 값만 허용하며 default는 disabled다. Phase 9의 testnet bootstrap은 공식 Spot Testnet endpoint만 고정해 사용하고 credential 기반 read-only 실행과 주문 실행을 분리한다. Phase 13 actual Testnet Case 2는 local in-scope gate 통과 후만 ETHUSDT에 별도 opt-in하고, 일반 Phase 13 ceiling은 각 신규 BUY decision notional 100 USDT 이하다. 다만 §16.20 Session 3 current-source target은 사용자가 승인한 10 USDT 이하로 더 좁히며, public Case 2 permission 경계가 10 USDT를 넘는 설정을 client 조립 전에 거부한다. 이 상한은 신규 노출 사전 추정치이므로 거래소 filter 검증을 대체하지 않는다. STOP/recovery SELL은 Position-free ETH·최신 filter 안의 정확한 보유 수량으로 제한하고 BUY cap을 적용하지 않는다. 각 BUY/SELL prepare는 fresh exchangeInfo, signed myFilters와 public referencePrice를 다시 읽는다. Base MAX\_ASSET은 최종 제출 quantity로 평가하고 MARKET MIN\_NOTIONAL/NOTIONAL만 reference price notional로 재검증한다. 공식 환산 가격식이 없는 quantity 기반 MARKET의 quote MAX\_ASSET이 있으면 고정 fail closed한다. MAX\_POSITION이 있거나 reference price를 authoritative하게 읽지 못하면 신규 BUY를 열지 않는다. live는 ADR-003의 release 승인, 매 실행 확인, configured policy의 version-nullable 세 상한-REALIZED\_ONLY·CANCEL\_AND\_LIQUIDATE와 reconciliation을 모두 통과해야 한다. Phase 13 구현 여부와 관계없이 별도 사용자 live 승인 record와 서명된 release checklist가 생기기 전에는 configuration, backend와 UI 세 경계에서 계속 비활성이다.

프로젝트 자체는 비공개·개인용으로 배포 license를 부여하지 않는다. Python의 LicenseRef-Proprietary-Private :: Do Not Upload, npm의 private: true·UNLICENSED, Cargo의 publish = false·repository LICENSE를 local supply inventory가 세 manifest와 root notice byte에 결합한다. Backend notice와 README도 같은 정책을 고지한다. Project 자체 UNKNOWN license group은 제거됐지만 retained scan은 current manifest bytes를 결합하지 않아 current\_vs\_scanned\_match=false다. PyInstaller 계열의 non-standard, GPL-2.0 두 third-party group과 raw license scan·SBOM-release artifact binding은 NO\_G0이며 dependency license·notice 의무는 계속 준수한다. 24시간 이상 Testnet soak는 Phase 13에서 영구 제외했으며 실행 또는 PASS로 표시하지 않는다. Exact lockfile-dependency metadata를 외부 OSV 서비스로 전송하는 것은 영구 불허다. scripts/run\_phase13\_offline\_osv.py의 OS network deny와 scanner offline cached DB만 허용하고 최신 advisory 확인 불가 또는 local cache 부재는 fail closed한다. 30분 raw OLS slope는 raw\_ols\_slope / candidate\_price \* 100, %/30분 봉으로 확정한다.

Testnet account stream 재연결은 첫 full account REST snapshot을 적용한 뒤 signed stream subscribe ACK를 먼저 확보한다. 그 ACK 아래에서 open/recent/same-ID 주문을 조화·적용하고 stream accumulator를 rebase한 뒤 두 번째 full account snapshot을 적용한다. barrier 중 event backlog가 생기거나 설명되지 않은 app-prefix 주문/fill, provenance 누락, balance 불일치가 발견되면 subscription을 닫고 command를 계속 차단한다. 복구가 성공하면 재조정 commit부터 publication까지 하나의 application RLock 구간을 유지한다. 두 REST snapshot으로 확정된 authoritative Account를 먼저 게시하고, 이어 다시 열린 command gate와 trading lifecycle을 게시한다. 이 publication이 실패하면 backend만 주문 가능 상태로 남기지 않고 event runtime failure로 영구 fail closed한다.

현재 process가 모르는 app-prefix 주문의 execution report는 정상 callback으로 버리거나 새 aggregate로 추측하지 않는다. 즉시 command gate를 RECONCILIATION\_REQUIRED로 닫고 authoritative trading snapshot을 게시한 뒤 같은 account recovery worker를 깨워 open/recent order와 두 account snapshot barrier를 다시 수행한다. 반면 app prefix가 없는 수동·외부 실행은 app-owned REST recovery로 귀속할 수 없으므로 process-lifetime blocker로 고정한다. 이 branch는 worker를 깨우지 않고 direct reconnect도 거부하며, fresh process의 account-wide 검증 전까지 command를 다시 열 수 없다.

Fresh process는 [ADR-007](#)에 따라 열린 앱 Position 이후의 외부 SELL만 복구한다. `listAccountExecutionsSince(symbol, orderId)`로 전체 client 주문-필을 두 번 대조하고, 두 Account의 ETH free/locked와 전체 open order 부채를 확인한 뒤 `recordReconciledExternalTrade(trade)`의 기존 저장-성과-게시 경로에 반영한다. Python operation은 각각 `list_account_executions_since`, `record_reconciled_external_trade`다. 실행 중 재연결이나 전략 주문 Action으로 이 경로를 호출하지 않고, 완료 후 `NOT_STARTED`를 유지한다.

`ACCOUNT_STREAM_UNKNOWN_OR_EXTERNAL_EXECUTION` category 자체는 일반 account disconnect와 app-prefix unknown도 함께 표현하므로 언제나 process-lifetime blocker인 것은 아니다. 별도의 prefixless external execution flag가 선 경우에만 이 category를 fresh-process 전용 blocker로 해석한다. Event runtime failure와 process ownership ambiguity도 각각 별도 process-lifetime flag로 보존한다. 이 세 flag 중 하나라도 선 process는 startup reconciliation 입구에서 외부 조회 전에 차단하고, startup의 최종 commit 직전에 다시 검사하며, startup 우회 direct `startTrading`도 pre-I/O 차단한다. 따라서 re-entrant callback이 final commit 사이에 flag를 세워도 startup 완료나 새 session을 게시할 수 없다.

Phase 13 actual failure finalizer는 exception-bound frozen cause snapshot을 시작 직전 `getReconciliationCauseSnapshot()`과 대조한다. 두 snapshot이 다른 category를 선택하지 않고 `CONFLICT`로 fail closed한다. 새 failure artifact writer는 schema v2만 쓰며, `first_cause`에 `reconciliation_required`, latch status와 `EXACT`일 때만 non-null인 category를 보존한다. V2 submission attempt는 sequence-symbol-side-order type-attempt number·timestamp만 보존하고 raw intent/client ID는 in-memory recovery matching 밖으로 내보내지 않는다. Historical schema v1 FAILED artifact는 v1 exact branch로 검증하고 v2 field를 소급 추가하지 않는다. Actual Testnet factory는 explicit evidence clock을 REST adapter의 `clock-result_clock`, permission proxy와 runtime/event stream에 동일 객체 identity로 전달한다. Evidence clock은 run 시작 시 wall UTC를 정확히 한 번 anchor한 뒤 `monotonic_ns` 경과만 더하므로 OS wall clock 역행의 영향을 받지 않는다. Harness가 각 REST 호출 직후 기록하는 top-level preflight 다섯 `observed_at`, permission attempt, runtime/account/UI와 failure-`NO_SIGNAL.SUCCESS` final timestamp, startup account source는 local run-scoped 축이다. 이 축 안에서는 모두 local `[started_at, completed_at]` 범위와 causal order를 지킨다.

`submit_time_filter_evidence`, Binance Kline source, server-aligned order attempted/result와 fill/Trade는 Binance server/exchange 축이다. 이 값은 local run window 또는 UI/final timestamp와 cross-axis 대소를 비교하지 않는다. 대신 같은 축 안의 fetch order, prepared composite 30초 freshness와 attempt → fill → result/Trade identity 인과를 강제한다. `SUCCESS`와 `NO_SIGNAL` validator는 이 server/exchange 축 전체를  $\pm 60$ 초 coherent shift해도 같은 의미로 통과한다. 이는 [Binance Spot REST Timing security](#)가 signed timestamp를 `serverTime-recvWindow` 축에서 검증하는 공식 계약과 일치한다. Wall-clock backward-regression 회귀는 trace의 local timestamp 범위를 보존하고 failure artifact seal을 검증한다. V3 public account의 첫 행은 `2 / ACCOUNT_SNAPSHOT_APPLIED`이며 run UUID-startup Account version을 source ID에 결속한다. Startup version은 `1` 이상이어야 하고 source 시각은 다섯 preflight observation 중 첫 시각보다 늦을 수 없다. Preflight가 읽은 Account version과 같은 public row가 반드시 있어야 하며, `SUCCESS`는 recovery가 확인한 effective free quantity 이상을 fingerprint version 또는 그 이후 Account row의 free quantity로 증명한다. 후속 `2.2.1 / ACCOUNT_POSITION_APPLIED`는 source time을 단조 증가시키고 Account version을 엄격히 증가시키며, 생성한 `ACCOUNT_UPDATED` envelope의 event ID-`published_at`-aggregate version과 정확히 일치해야 한다. Harness는 startup maximum 이하의 buffered/replayed Account DTO를 증거로 추가하지 않는다. Startup 뒤 전진한 모든 `ACCOUNT_UPDATED`와 public account row는 동일한 exact subsequence여야 하므로 어느 방향의 누락·추가도 허용하지 않는다. Transport `published_at`은 전역 단조이고 aggregate version은 동일 version 재발행만 허용하며 감소할 수 없다. `ORDER_EXECUTED`/ `PERFORMANCE_UPDATED`는 preflight verification 후에만 발행된다.

Account와 Market controller는 version-immutable state-source provenance를 함께 반환하는 state pointer/snapshot을 공개한다. Startup account evidence producer는 application lock에서 Account state를 한 번 읽은 뒤 같은 run-scoped evidence clock을 읽어 state → clock 인과를 원자적으로 고정한다. Market observer도 같은 lock에서 state와 evaluation context를 한 번만 freeze한다. `NO_SIGNAL` collector는 별도 synthetic market snapshot을 만들지 않고 실제 production `1L.1 / KLINE_OBSERVED` frozen event만 수집하며, cursor는 그 event가 결속한 captured market version으로만 전진한다.

Current full trace schema v3는 `SUCCESS`와 `NO_SIGNAL` 두 outcome만 허용한다. Public Action을 이미 관찰한 뒤 blocker가 생기거나 `BLOCKED/UNKNOWN/FAILED`가 되면 trace shape를 추정하지 않고 failure evidence schema v2로 봉인한다. V3 `NO_SIGNAL` market evidence는 실제 production `1L.1, KLINE_OBSERVED, TYPE_0`이고 action/evaluation/side/strategy가 모두 null인 행만 허용한다. V3 `NO_SIGNAL`의 각 `source_event_id`는 trace 전체에서 유일해야 하며, market/context version만 증가시켜 같은 source를 중복 재생한 행도 위조로 거부한다. `SUCCESS`는 evaluation ID `market:{market_version}:{source_event_id}`와 parsed source identity/time을 exact 결속한다. Action 단일 source는 30m open/closed 또는 closed 1m만 허용한다. Atomic source는 ordered (1m closed, 30m closed) 뒤 UTC 4h 경계에서 (4h closed, 4h open), UTC 자정에서 (1d closed, 1d open) pair를 더한 정확히 2/4/6개 batch이며 모든 pair는 같은 boundary에 도달해야 한다. `NO_SIGNAL`은 production이 게시할 수 있는 단일 4h/1d 관찰도 허용한다. Market version은 엄격 증가하고 context version은 감소하지 않는다.

V3 `SUCCESS`의 policy version은 `13`, 모든 regime은 `TYPE_0`, BUY/SELL configured cap은 같고 client order ID는 `bat-{sha256(session_id + NUL + intent_id)[:24]}-0`이다. SELL intent는 `force-sell:{session_id}`, STOP evaluation은 `stop-{session_id}-phase13-stop-{run_id}`로 exact 결속한다. 각 order trace의 message 1은 `v→v`, message 2는 같은 before에서 `v→v+1`, message 14는 `order-outcome-{client_order_id}-CASE_C_POSITION_OPENED/FORCE_SELL_FINISHED`다. BUY message 2의 `context_version_after`는 immutable evaluation fingerprint 및 `1L.3`에 기록된 context version과 정확히 같아야 한다. 첫 비동기 message 8 또는 9 전의 모든 non-final command ID는 evaluation ID이고, 그 경계부터는 intent ID다. 즉 immediate fill의 suffix까지 evaluation ID를 유지하고, 비동기 경계가 생길 경우에만 한 번 intent ID로 전환하며 다시 되돌릴 수 없다. Final message 14는 위 outcome event identity를 유지한다.

Messages `1..6.1`의 `exchange_order_id`는 반드시 null이고, concrete exchange ID는 message 7 또는 9에서만 처음 관찰할 수 있다. 한번 관찰한 뒤에는 이후 message와 result/Trade에서 null로 돌아가거나 다른 값으로 바뀔 수 없다. Message 9는 REST query 없이 독립적으로 도착하거나 반복될 수 있고, User Data Stream partial 9 뒤 scheduled query `8/8.1/8.2`가 terminal을 확정하는 형태도 유효하다. Same-ID REST query 예산은 주문당 최대 4회지만 stream message 수에는 임의의 4회 제한을 두지 않으며, 전체 trace 구조 상한 2048개만 적용한다. 이는 주문 갱신을 `executionReport`로 제공하는 공식 [Binance User Data Stream](#)과 한 주문에서 여러 `PARTIALLY_FILLED` event 뒤 terminal event가 올 수 있음을 설명하는 공식 [Binance Market Orders FAQ](#)에 맞춘다. Result와 durable Trade는 extra `UNKNOWN` 없이 attempt와 zip된 BUY → SELL exact order이며 Trade ID는 `trade-{exchange_order_id}`다. 각 result의 fill은 (event\_time, canonical integer tradeId) strict ascending이고 Binance `tradeId`는 non-negative여야 한다. 이는 allocation과 실제 trade를 구분하는 공식 [Binance SOR FAQ](#)의 계약과 일치한다. Fee asset은 ETH 또는 USDT만 허용하고, quote fee는 USDT fee 자체 또는 Decimal precision 34의 `ETH fee * price`로 다시 계산한다. Fill fee 합은 Trade fee와 같아야 하며 BUY cost basis-SELL net proceeds-exact quantity allocation으로 domain realized PnL을 다시 계산해 Trade와 Performance run/total에 결속한다. Empty baseline의 realized PnL과 fee는 모두 zero다. Production Position 회계가 지원하지 않는 양수 ETH base-fee SELL은 산술을 맞춰도 `SUCCESS`로 봉인하지 않는다. Preflight MARKET BUY received-asset commission rate가 zero이고 discount asset이 없으면 BUY result의 모든 fill commission과 durable Trade의 `fee_amount-fee_quote_amount`도 정확히 zero여야 한다. 이는 MARKET BUY commission을 수신 asset quantity와 buyer/taker rate에 적용하고 discount를 별도로 정의하는 공식 [Binance Commission FAQ](#)에 따른다.

V3 transport sequence는 정확히 `1..N`이다. Account event가 Trade cycle 사이에 interleave하는 것은 허용하지만 `ACCOUNT_UPDATED.related_id`는 null이어야 한다. 각 durable Trade는 adjacent `ORDER_EXECUTED` → `PERFORMANCE_UPDATED` pair와 exact order로 대응하고, 그 pair 뒤 다음 order 전에는 trace context version 이상인 `TRADING_SESSION_UPDATED`가 있어야 한다. 두 cycle의 session `related_id`는 같은 canonical UUIDv4다.

`SUCCESS` STOP SELL의 account filter-symbol rule-account-wide open state-reference-price 관찰은 모두 authoritative BUY terminal result 이후에 시작해야 한다. 또한 [Binance Spot New Order](#)의 FULL `transactTime`와 All Orders의 order query `updateTime/time` 중 하나가 없는 성공 payload는 local clock으로 추정하지 않고 `UNKNOWN` reconciliation으로 fail closed한다. Known-safe failure recovery의 성공-skip-실패 판정 뒤 모든 outcome에서 logical submission permit을 달고, scheduler seal보다 먼저 재개 불가능하게 만든다. 이어 `sealEventRuntimeFailureGate()`가 같은 session lock에서 terminal check와 active/reconciliation blocker commit을 한 번에 수행한다.

**TERMINATED/NOT\_STARTED**는 변경하지 않고 기존 **RECONCILIATION\_REQUIRED**에는 event blocker만 유지한 채 cause를 다시 기록하지 않아 성공한 STOP recovery의 zero-exposure와 historical first-cause latch를 보존한다. Atomic seal이 공개 status를 담은 뒤 Context publication에서 예외가 나면 stable **FAILURE\_SCHEDULER\_GATE\_PUBLICATION\_FAILED**를 남기고 primary close-fresh read-artifact seal을 계속한다. 이 secondary code는 completed recovery와 함께 쓸 수 없고 runtime **RECONCILIATION\_REQUIRED**에만 허용한다. V2 runtime state는 **RUNNING/STOPPING**을 거부하며, fresh **verified\_at**은 마지막 submission attempt 이 상이다. Runtime durable count와 verified run/durable count는 attempt 수 이하이고, verified 두 count는 같아야 한다. Reconciliation status는 concrete blocker **false**와 함께 쓸 수 없고, verified durable suffix 0은 zero Position에만 계속한다.

Fresh verification은 runtime creation, startup application, market, regime, account, history, reconciliation, stream, filter, local state, open orders, open order lists, recent orders, durability, cleanup의 stable stage 중 최초 실패를 고정한다. Fresh runtime에는 source history, pending sidecar와 manual-kill control을 직접 전달하지 않고, filesystem root부터 최종 parent까지 모든 component를 **0\_NOFOLLOW**로 연 owner-only source/destination **0700** FD-identity-ctime chain을 fresh runtime 종료까지 함께 pin한다. No-follow leaf descriptor-exclusive-**0600**-fsync로 복사한 isolated durability snapshot만 전달한다. Finalizer는 두 ancestor chain 전체의 identity/ctime과 source/copy leaf의 device/inode/owner/mode/link/size/SHA-ctime-mtime before-after fingerprint, account-wide open orders/order lists, symbol open results, recent baseline, stream-reconciliation를 publication 직전 다시 읽는다. Final leaf 검증은 source → copy → source 순서로 캡처해 copy 확인 사이의 source ABA도 거부하고, 그 뒤 ancestor chain을 한 번 더 검증한다. Destination/source ancestor descriptor chain은 어느 close가 실패해도 양쪽 전체 close를 계속 시도하고 최초 cleanup error를 보존한다. Artifact publication도 file close, 임시 leaf unlink, directory close를 모두 best effort로 시도하고 최초 cleanup error를 유지한다. **evidence\_errors**는 cause → recovery → scheduler seal → runtime field → fresh primary → runtime close → durability → snapshot cleanup → final-verification downgrade의 생성 순서를 보존하고, no-attempt concrete Position-matching은 zero이며 serial runtime/fresh pending은 한 건을 넣을 수 없다. 같은 inode의 content 복원과 absent sidecar 생성·삭제 ABA도 fail closed한다. 어느 하나라도 변하거나 불안전하면 **VERIFIED**를 게시하지 않고 최초 stage와 stable typed reason으로 **INCOMPLETE**를 봉인한다. Ancestor ctime pin은 같은 evidence directory의 무관한 sibling 생성·삭제도 변조로 감지해 fail closed한다. 따라서 actual execution 동안 source와 destination evidence directory를 격리하고 sibling mutation을 금지해야 하며, 동시 변경이 있으면 재시도 없이 **DURABILITY/INCOMPLETE**로 종료한다.

위 immutable snapshot과 exact V3 chronology/accounting/outcome-routing 보장은 local-only다. 이번 보강에서 Keychain lookup, Binance signed/public target, Testnet diagnostic/actual, submission attempt, order POST, BUY-STOP SELL과 durable Trade는 모두 **0**건이며 current source의 external binding은 아직 없다. 특히 current failure-evidence v2 writer는 external target과 미결속이다. 아래 수치는 이 추가 보강 전 확정된 historical local gate이며 최종 전체 회귀 수를 뜻하지 않는다. Credential/order opt-in을 제거한 당시 local gate는 Backend **962**개 **OK (skipped=8)**, scripts **183/183**, secure runner **14/14**, cause integration **50/50**, Case 2 local helper **29/29**와 external actual **1** safe skip, convention **2/2**, Communication **126/126**을 통과했다. 추가 보강 뒤 current Backend **967**개는 모두 통과하고 external 여덟 건을 safe skip했다. Root scripts **183**개 중 세 건은 retained Phase 12 app의 exact historical DMG가 누락된 local supply pair를 fail closed했다. 이는 external release binding이 아니므로 actual Phase 13과 live readiness는 계속 **NO\_G0**이고 별도 live 승인 전 **live**는 disabled다.

10.4 변경 Operation 추적성

| 결정/메시지                   | Caller -> Owner                        | 확정 Operation                                                                        |
|--------------------------|----------------------------------------|-------------------------------------------------------------------------------------|
| D-03 / 1.5.1             | RegimeController -> RegimeSTM          | handle(event, context?) : RegimeSTMResult                                           |
| D-02 / 6.1.1             | UIStateController -> RegimeController  | setRegimeType(regimeType, commandId, expectedVersion) : TradingLogicSelectionResult |
| D-02 / 6.1.1.1           | RegimeController -> TradingController  | fetchSelectedTradingLogic(regimeType) : TradingSTM                                  |
| D-02 / 6.1.1.1.1         | TradingController -> TradingSTM        | getSTMInstance(regimeType) : TradingSTM                                             |
| D-15 / 7.1.1             | UIStateController -> TradingController | startTrading(commandId, expectedVersion) : TradingSessionResult                     |
| D-02 / G-07              | TradingController -> TradingSTM        | handle(UPPER_BAND_TOUCHED, context) : TradingSTMResult                              |
| D-05 / 8.1.1             | UIStateController -> TradingController | stopTrading(commandId, expectedVersion) : TradingSessionResult                      |
| D-05-D-08 / 8R.1.1       | UIStateController -> TradingController | liquidateRecoveredPosition(commandId, expectedVersion) : TradingSessionResult       |
| D-05-D-08 / 8R.1.1.2     | TradingController -> TradingSTM        | handle(STOP_CONFIRMED, recoveredContext) : TradingSTMResult                         |
| D-05 / 8.1.1.1           | TradingController -> TradingSTM        | handle(STOP_CONFIRMED, context) : TradingSTMResult                                  |
| D-05 / 8.1.1.2b          | TradingController -> APIGateway        | sellAllPosition(symbol, quantity) : OrderResult                                     |
| D-08 / 8.1.1.2p-Case 2 8 | TradingController -> APIGateway        | queryOrderResult(symbol, orderId?, clientOrderId?) : OrderResult                    |
| D-08 / 8.1.1.2p.1        | TradingController -> APIGateway        | cancelOrder(symbol, orderId?, clientOrderId?) : OrderResult                         |

| 결정/메시지                        | Caller -> Owner                                                   | 확정 Operation                                                                                                                             |
|-------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| D-08 / startup 복구             | TradingController -> APIGateway                                   | listOpenOrderResults(symbol) : List<OrderResult>                                                                                         |
| D-08 / startup 복구             | TradingController -> APIGateway                                   | listRecentOrderResults(symbol, limit) : List<OrderResult>                                                                                |
| D-08 / stream 재연결             | TradingController -> WebSocketGateway/APIGateway                  | reconnectAccountStreamAfterReconciliation(recoveryCommitObserver : Subscription)                                                         |
| D-04 / Case 2 14, 8.1.1.3     | TradingController -> TradingSTM                                   | orderFinished(event, context) : TradingSTMResult                                                                                         |
| P13-01 / Case 2 5.1           | TradingController                                                 | evaluateBuyRisk(order) : RiskDecision                                                                                                    |
| P13-01 / manual kill recovery | TradingController -> TradeHistoryController/APIGateway/TradingSTM | setManualKill(...), resumeManualKillCleanup(), saveManualKillControlState(state), getManualKillControlReplay()                           |
| P13-03 / process ownership    | TradingController                                                 | markProcessOwnershipAmbiguous(reason) : void                                                                                             |
| P13-03 / stale owner release  | Tauri native startup -> runtime ownership artifact                | inspectStaleRuntimeOwner() : Attestation?, releaseStaleRuntimeOwner(attestation) : void                                                  |
| P13-04 / reconciliation cause | Phase 13 finalizer/event runtime -> TradingController             | getReconciliationCauseSnapshot() : ReconciliationCauseSnapshot, markEventRuntimeFailed() : void, sealEventRuntimeFailureGate() : Boolean |

| 결정/메시지                           | Caller -> Owner                                                       | 확정 Operation                                                                                                                                                                               |
|----------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <hr/>                            |                                                                       |                                                                                                                                                                                            |
| P13-04 / 1L.1~1L.3               | MarketDataController -> TradingController                             | observeMarketEvaluation(evaluation, marketVersion, sourceEventId) : TradingEvent?, readStateSnapshot() : MarketStateSnapshot                                                               |
| <hr/>                            |                                                                       |                                                                                                                                                                                            |
| P13-04 / actual preflight-submit | TradingController -> APIGateway                                       | fetchAccountRelevantFilters(symbol) : AccountRelevantFilters, hasAnyExchangeOpenOrders() : Boolean, hasAnyExchangeOpenOrderLists() : Boolean, fetchReferencePrice(symbol) : ReferencePrice |
| <hr/>                            |                                                                       |                                                                                                                                                                                            |
| P13-04 / failure finalizer       | Phase 13 actual harness -> TradingController/APIGateway/fresh runtime | _recordActualFailureEvidence(error) : void                                                                                                                                                 |

| 결정/메시지                                | Caller -> Owner                                                  | 확정 Operation                                                                        |
|---------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| P13-04 /<br>market full-<br>resync    | MarketDataController -> RegimeController                         | reconcileMarketStream() : MarketSnapshot, reconcileRegime(snapshot) : RegimeResult? |
| D-07 / 1.4                            | MarketDataController -> RegimeController                         | calculate4HIndicators(snapshot) : IndicatorSnapshot                                 |
| D-10~D-13 /<br>Case 2 13,<br>Case 3-4 | TradingController/UIStateController -><br>TradeHistoryController | 기존 recordOrderExecution, getTradeDetails, exportCSV                                 |
| D-14 / process<br>boundary            | BackendUiAdapter -> transport route -> 기존<br>Controller          | ADR-005의 /v1/* HTTP/WS 계약                                                           |

## 10.5 기존 클래스 우선 검토 결과

- 주문 취소와 open/recent order 조회는 **APIGateway**의 Binance REST 캡슐화 책임에 응집되므로 이 클래스에 추가했다.
- Account relevant filter 세 scope, account-wide open order/list empty 조회와 MARKET **MIN\_NOTIONAL/NOTIONAL** 용 reference price 조회도 같은 Binance REST 캡슐화 책임에 응집되므로 새 business lifeline 없이 **APIGateway**의 immutable adapter DTO로 추가했다. Quote **MAX\_ASSET** 환산은 이 DTO 책임으로 추정하지 않는다.
- Regime evaluation Action dispatcher는 **RegimeController**, Trading event loop와 retry scheduler 정책은 **TradingController**의 private 구현이다. Production bootstrap의 단일 interruptible worker는 Controller의 bounded cycle을 호출하고 상태 publication을 연결할 뿐 due 시각, same-ID, 잔여 수량과 retry 예산을 판단하지 않는다.
- JSONL은 **TradeHistoryRepository**, CSV write는 **CSVFileGateway**, transport DTO 변환은 **BackendUiAdapter**와 함수 기반 route가 소유한다.
- 새 trading strategy class는 만들지 않는다. 새 REGIME logic은 Event-Action Table과 기존 **TradingSTM** registry 선택 구조를 먼저 확장한다.
- process boundary adapter인 **BackendUiAdapter** 외에 새 Communication 업무 클래스는 Phase 0에서 승인하지 않았다.

## 10.6 명세 확인

- ☒ canonical REGIME과 다섯 mapping 상태가 명시되었다.
- ☒ 메시지 1.5.1과 클래스 8.13의 RegimeSTM signature가 일치한다.
- ☒ Case 2 14와 클래스 8.5의 concrete order outcome signature가 일치한다.
- ☒ stop의 무포지션·보유·pending branch와 완료 조건이 명시되었다.
- ☒ 기존 27개 클래스에 Operation을 우선 배치하고 새 strategy class를 만들지 않았다.
- ☒ Phase 9 Testnet 주문 adapter는 별도 이중 opt-in-상한 아래 포함했고 credential 값과 live 활성화는 포함하지 않았다.
- ☒ Phase 6에서 **TYPE\_0**의 109개 lower-BB registry와 G-07 안전 종료를 **READY**로 고정했고 **TYPE\_1~TYPE\_4**는 typed failure로 거부했다.
- ☒ Registry **READY**와 실행 mode-account stream readiness를 결합한 **command\_enabled** gate를 분리했다.
- ☒ Configured-unbounded 세 상한, **REALIZED\_ONLY**, **CANCEL\_AND\_LIQUIDATE**, 일반 Phase 13 **ETHUSDT BUY 100 USDT** outer ceiling과 Session 3 public Case 2의 **10 USDT** 축소 실행 cap을 명세에 동기화했다.
- ☒ 30분 EMA9/SMA9 seed/alpha 0.2/최근 6개 OLS/Decimal 34/최종 HALF\_EVEN과 **raw\_ols\_slope / candidate\_price \* 100.%/30분봉**을 확정했다. Concrete builder는 기존 seam의 application 구현 세부사항으로 기록하고 architecture audit 대상에 포함했다.
- ☒ Public 1L.1~1L.3과 Case 2 전체를 local deterministic/production REST memory transport로 검증하고, actual current-host Tauri picker의 선택·취소 terminal 계약까지 확인해 Communication manifest 126/126을 완료했다. Actual Phase 13 Testnet 주문 증거는 별도 readiness 경계다.
- ☒ 외부 OSV 서비스 전송을 영구 불허하고 OS network deny-offline cached DB-fail-closed 최신성 경계를 명세에 동기화했다.
- ☒ 24시간 soak 영구 제외를 PASS 증거와 구분했다.